The Vehicle Plant in Port-Hamiltonian Form: Theory, Implementation and Validation for GRL-SNAM

Joe Rivera
Computational Visualization Center
The University of Texas at Austin

October 2026; revised November 2026
libcvc 3.7.1 (`transfix/libcvc`) · GRL-SNAM `main` at `0e21085` (`CVC-Lab/GRL-SNAM`)

Abstract

GRL-SNAM navigates by learning the coefficients of a potential field and driving a vehicle with a bicycle controller. The vehicle under that controller is now a dynamic single-track model with Pacejka tires, load transfer and actuator lag, implemented in libcvc and twinned in GRL-SNAM's torch code. This paper writes that model as a port-Hamiltonian system on $T^*SE(2)$ *exactly*, at every tire slip and not only in the linear-tire regime. On level ground it reduces to $\mathfrak{se}(2)^*$, with the pose recovered by reconstruction. The dissipation is the tires' secant stiffness, which is positive semi-definite even when a tire slides. The port is the pair of axle forces, with the wheels' ground speeds as its conjugate output. Steering is not a port: it only turns the directions along which the tires push, so it does no work. Each tire's slip is measured in its wheel's frame, so the secant form leaves no remainder, and gravity on a grade is booked as a power line of its own. The integrator's step keeps per-step energy books whose residual is second order in the step, and an opt-in discrete-gradient step closes them exactly. libcvc's four-wheel vehicle with suspension has the same form, with one term the single track lacks: the moments of the tires' lateral forces on roll and pitch, a one-way coupling that is booked on its own line. Opt-in extensions put wheel spin, with a (wheel torque, wheel speed) port, compliant contact, with a barrier energy in H, and the slope across the heading, with gravity in H, into the same form. We also describe the learned stochastic port-Hamiltonian plant on $SE(2)$ (`cvc::ph`) and its energy balance, which includes the noise's power. We also describe an opt-in form of that plant in which steering is a modulation, so that only the drive supplies power, as in the analytic plant. We give the navigator's per-step energy identity, with its context and projection work at each tick. Every claim is tied to a test in libcvc or GRL-SNAM, with the tolerance the test enforces and the value measured on the released code (Section 11), so it can be checked independently.

Contents

1 Introduction **3**

2 Preliminaries **5**
2.1 Port-Hamiltonian systems 5
2.2 Planar rigid bodies and the Lie–Poisson structure 5
2.3 Where the plant sits in GRL-SNAM 5

3 The dynamic single-track vehicle **5**

4 The port-Hamiltonian form **7**
4.1 The tires' secant stiffness . . . 7
4.2 The structure . . . 7
4.3 Steering is not a power port . . . 9
4.4 The speed floor . . . 9
4.5 Grade . . . 9
4.6 Tangent versus secant . . . 10
4.7 The kinematic limit and the Jacobi identity . . . 10
4.8 Externally defined ports . . . 11

5 The discrete step and its energy books **11**
5.1 The step . . . 11
5.2 The books . . . 12
5.3 Closing the books exactly . . . 13
5.4 The live plant . . . 14

6 The four-wheel vehicle **15**

7 Wheel spin, the side slope and compliant contact **16**
7.1 Wheel spin . . . 16
7.2 The full form and the side slope . . . 18
7.3 Compliant contact . . . 19

8 The learned plant: `cvc::ph` **20**
8.1 Model . . . 21
8.2 Gaussian processes on random Fourier features . . . 21
8.3 Energy balance, output and noise power . . . 21
8.4 The step . . . 22
8.5 Training . . . 22
8.6 Steering: a port in the reference form, a modulation by option . . . 23
8.7 The potential's form, and the wheels as inputs . . . 24

9 The navigator's energy identity and context work **24**
9.1 The per-step identity . . . 24
9.2 Two checks . . . 25
9.3 Context work and projection work . . . 26
9.4 Two sets of books, not one . . . 26

10 GRL-SNAM's side **26**
10.1 The twins and their parity tiers . . . 26
10.2 What is not twinned yet . . . 27
10.3 Using the plant and its books from GRL-SNAM . . . 27

11 Validation **30**

12 Limitations and future work **45**
12.1 Gaps raised in review, and the work on them . . . 46

A Reproducing the validation tables **47**
A.1 What was run 47
A.2 libcvc 48
A.3 What the libcvc tests print 48
A.4 GRL-SNAM 53

B Equation-to-code map **53**

1 Introduction

GRL-SNAM [1] treats navigation as motion in a learned energy landscape. A small network predicts the coefficients $(\alpha_j, \beta, \gamma)$ of a potential: barrier weights per obstacle, a goal spring and a damping. A controller turns the potential's force into steering and acceleration commands for a vehicle. GRL-SNAM's own guide to the C++ port (`docs/cvc-nav-and-grl-snam-guide.md`, "A note on the vocabulary") is careful about what this does and does not buy. The deployed navigator is a learned-coefficient potential field with viscous damping, on top of a kinematic bicycle and a pure-pursuit controller. No Hamiltonian is carried through it, and no energy or passivity argument transfers to it.

This paper is about the layer under the navigator: the vehicle. Since libcvc 3.7.0, a dynamic single-track vehicle (the plant) can replace the kinematic update where a policy asks for it. libcvc 3.7.1 writes that vehicle as a port-Hamiltonian (pH) system with a genuine Hamiltonian H, momenta, a positive semi-definite dissipation, and a power port. It keeps books of where the vehicle's kinetic energy goes, step by step. The same release adds two related pieces: a learned stochastic pH model of a vehicle, and per-step energy books for the navigator's point-mass surrogate. Across the navigator's ticks those books include the potential's changes of context.

Contributions.

1. An exact pH form of the dynamic single-track vehicle, valid at every slip (Proposition 1). The dissipation is built from the tires' secants, the port is the axle forces, and steering is shown not to be a power port (Section 4). The same form, with its books, for the four-wheel vehicle with suspension, and an opt-in grade force that puts the terrain's energy in H (Section 6). Opt-in wheel spin, compliant contact and the slope across the heading, in the same form and with their books (Section 7).

2. The discrete step's energy books, with a residual that is second order per step (Section 5), an opt-in discrete-gradient step whose books close exactly (Section 5.3), and a live plant that keeps them for every vehicle in a navigation world.

3. The learned plant `cvc::ph`: its structure, its energy balance including the noise power, its integrator and its training objective (Section 8). It also has an opt-in form in which steering modulates the model instead of supplying power, as it does in the analytic plant (Section 8.6).

4. The navigator's per-step energy identity, the two checks built on it, and the context work at tick boundaries (Section 9).

5. GRL-SNAM's side: the torch twins and their parity tiers, what is not twinned yet, and how a GRL-SNAM rollout or trainer can use the plant and its books (Section 10).

6. A validation table that maps each claim to a test, its tolerance, the measured value and the commit that added it (Section 11).

What is not claimed. The closed loop of navigator, controller and plant is not port-Hamiltonian. The controller is a path follower, not an energy-shaping controller (Section 12). Neither the analytic plant's default integrator nor the learned plant's is structure-preserving. Both keep books with a residual instead; an opt-in step closes the analytic plant's exactly (Section 5.3). The learned plant's filter-based evidence bound is not yet implemented. The four-wheel vehicle's pH form (Section 6) is passive with respect to its wheels' forces except for a one-way lever term, and only its wheel-spin form has a GRL-SNAM twin. Wheel spin, compliant contact and the side slope are opt-in (Section 7): the plants' defaults, and every result outside that section and its tables, are without them. The four-wheel plant has no compliant contact.

Versions. Every statement about code refers to libcvc 3.7.1 or later and GRL-SNAM `main` at `0e21085`, unless it names a later pull request.

- The dynamic plant itself is transfix/libcvc #714, released in 3.7.0.
- The learned plant is #758, released in 3.7.1.
- The pH form and the books are #831, #832 and #833, also released in 3.7.1.
- The front slip in its wheel's frame (Sections 3 and 4.4) is #852, merged as `140bc2d0` and released in 3.7.2. It changes the plant's outputs; its golden for the pH form is #848's.
- The four-wheel vehicle's pH form and books (Section 6) are #850, merged as `45dab152` and released in 3.7.2.
- Steering as a modulation of the learned plant (Section 8.6) is #851, merged as `b8ac9975` and released in 3.7.3.
- The live plant's ground and rigid-body world (Section 5.4) are #869, merged as `5ca26cf6` and released in 3.7.2, and its driver is #873, merged as `462d8229` and released in 3.7.3. Its `pycvc` binding is #979, merged as `781b9a2f`, and the binding of the options of Section 7 is #1002, merged as `1ded7bed`; both were first released in 3.7.9. #1038, merged as `2434c33a` and first released in 3.7.12, binds `contact_step_depth`.
- Wheel spin (Section 7.1) is #982, merged as `4b640693`; compliant contact (Section 7.3) is #983, merged as `2728bca8`; the full form and the side slope (Section 7.2) are #987, merged as `b89992ef`. All three were first released in 3.7.9. CVC-Lab/GRL-SNAM #146 (`b6ddb801`) twins them. The contact's implicit damper and its substeps are #1004, merged as `ce9aebaf` and first released in 3.7.12.
- The navigator's geometry rollout takes semi-implicit Euler by default since #981 (Section 9), merged as `45cb239a` and first released in 3.7.12.
- The opt-in discrete-gradient step (Section 5.3) is #978, merged as `789d2f6e` and first released in 3.7.12. Its wheel-spin version is #1045, merged as `fcfc4ee7`, in no release yet.

The measurements in Section 11 were taken on libcvc `master` at `c1f104c0`, whose `nav`, `ph` and `physics` sources and tests are identical to v3.7.1's, except for the rows from #850, #851, #852, #869, #873, #978, #981, #982, #983, #987, #1004 and #1045. Appendix A gives the commit each of those was measured on.

2 Preliminaries

2.1 Port-Hamiltonian systems

An input-state-output pH system [7] with state x, stored energy $H(x)$, input u and output y is

$$\dot{x} = (\mathsf{J}(x) - \mathsf{R}(x))\nabla H(x) + \mathsf{G}(x)\,u, \qquad y = \mathsf{G}(x)^\top \nabla H(x), \tag{1}$$

with $\mathsf{J} = -\mathsf{J}^\top$ (the interconnection) and $\mathsf{R} = \mathsf{R}^\top \succeq 0$ (the dissipation). Along solutions, $\dot{H} = -\nabla H^\top \mathsf{R} \nabla H + y^\top u \le y^\top u$, so the system is passive from the port (u, y), with H as storage. The structure tells you where energy goes: J moves it without loss, R dissipates it, and the port $y^\top u$ is the only way it enters from outside. If R may depend on the input, nothing in this changes, provided it is positive semi-definite for every input value.

2.2 Planar rigid bodies and the Lie–Poisson structure

A planar rigid body has pose $q = (x, y, \psi) \in SE(2)$ and body twist $v = (v_x, v_y, r) \in \mathfrak{se}(2)$: the velocity of its centre of gravity in the body frame, and its yaw rate. With mass matrix $M = \mathrm{diag}(m, m, I_z)$ the momenta are $p = Mv \in \mathfrak{se}(2)^*$, and the kinetic energy is $H(p) = \frac{1}{2} p^\top M^{-1} p$, so $\nabla_p H = v$. The body-frame equations of motion of a free body are the Euler–Poincaré equations $\dot{p} = \mathrm{ad}_v^*\, p + f$, with f the applied body wrench and

$$\mathrm{ad}_v^*\, p = \mathsf{J}(p)\, v, \qquad \mathsf{J}(p) = \begin{pmatrix} 0 & 0 & p_y \\ 0 & 0 & -p_x \\ -p_y & p_x & 0 \end{pmatrix}. \tag{2}$$

$\mathsf{J}(p)$ is skew-symmetric and linear in p: it is the Lie–Poisson structure of $\mathfrak{se}(2)^*$ [6], so it satisfies the Jacobi identity. With M diagonal and equal masses on v_x and v_y, $\mathsf{J}(p)v = (mv_y r, -mv_x r, 0)$. These are the familiar $+rv_y$ and $-rv_x$ terms of a vehicle's body-frame equations.

2.3 Where the plant sits in GRL-SNAM

GRL-SNAM's navigator evaluates a potential U_{nav} at the agent's position o,

$$U_{\mathrm{nav}}(o) = \tfrac{\beta}{2}\|o - g\|^2 + \textstyle\sum_j \alpha_j\, b(d_j(o)) + (\text{material and terrain terms}), \tag{3}$$

with b the IPC barrier of each obstacle's clearance d_j, and g the current goal or carrot. Damping $-\gamma v$ is dissipation, not part of U. An external force, where one is used, is a port, not part of U either. libcvc evaluates U_{nav} along every rollout (`inc/cvc/nav/detail/nav_potential.h`), and GRL-SNAM's torch twin (`grl_snam/nav_potential.py`, CVC-Lab/GRL-SNAM#133) is held to it. The controller (`bike_control` in `diff_rollout.h`; `sdf_nav.bike_control` in GRL-SNAM) turns the force $-\nabla U_{\mathrm{nav}}$, plus any external force, into a steering command δ_{cmd} and an acceleration command a_{cmd}. A plant then moves the vehicle. The default plant is the kinematic bicycle, $\dot{\psi} = (v/L)\tan\delta$. The dynamic plant of this paper takes the same commands, so a policy can train through tires, load transfer and actuator lag with the controller and its adjoint unchanged.

3 The dynamic single-track vehicle

The plant is `cvc::nav::diff::dyn_plant` (`inc/cvc/nav/detail/dyn_bike.h`). It is written once over a scalar type: float in training, double in the tests, and a forward-mode dual number for its adjoint. Its torch twin is `grl_snam.dyn_bike.dyn_plant`. The model is the standard single-track ("bicycle") vehicle [4] with Pacejka lateral forces [3]. We summarize what the pH form needs.

State and parameters. The state is $(x, y, \psi, v_x, v_y, r, \delta, a)$: the rear axle's position, the heading, the body twist at the centre of gravity, the road-wheel steering angle, and the drive or brake force per unit mass. The last two sit behind first-order actuators. The parameters are the mass m, the yaw inertia I_z, the distances l_f, l_r from the centre of gravity to the axles ($L = l_f + l_r$), the centre of gravity's height h_{cg}, the ground's friction μ, Pacejka factors B_f, B_r and C, drag c_d, rolling resistance c_{rr}, the drive and brake split between the axles, and a speed floor $\varepsilon > 0$.

Loads and grip. Quasi-static longitudinal load transfer gives axle loads $F_{z,f} = m(gl_r - ah_{\mathrm{cg}})/L$ and $F_{z,r} = m(gl_f + ah_{\mathrm{cg}})/L$, each floored at a small fraction of mg. The drive or brake force ma splits between the axles, and each share is saturated smoothly at $\mu F_{z,w}$. These are the axle forces $F_{x,f}, F_{x,r}$. What grip they leave for cornering, P_w, follows from the friction ellipse with the longitudinal force first. Both depend on the actuator state a.

Directions. Each tire's lateral force reaches the twist's coordinates (v_x, v_y, r) along n_w, and each axle's drive or brake force along t_w:

$$n_f(\delta) = \begin{pmatrix} -\sin\delta \\ \cos\delta \\ l_f\cos\delta \end{pmatrix}, \quad n_r = \begin{pmatrix} 0 \\ 1 \\ -l_r \end{pmatrix}, \quad t_f(\delta) = \begin{pmatrix} \cos\delta \\ \sin\delta \\ l_f\sin\delta \end{pmatrix}, \quad t_r = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}. \tag{4}$$

So $n_w^\top v$ is the sideways velocity of that axle's contact patch in its wheel's frame, and $t_w^\top v$ is its ground speed along the wheel.

Slips. Each slip is measured in its wheel's frame, as in the four-wheel model. With $w_w = n_w^\top v$ the axle's sideways velocity there, $w_f = \cos\delta\,(v_y + l_f r) - \sin\delta\, v_x$ and $w_r = v_y - l_r r$, and $u_w = \sqrt{(t_w^\top v)^2 + \varepsilon^2}$ its floored forward speed, the slip angles are

$$\alpha_w = -\operatorname{atan2}(w_w, u_w), \qquad w \in \{f, r\}. \tag{5}$$

The floor keeps them smooth through zero speed. At the rear, $u_r = \bar{u} = \sqrt{v_x^2 + \varepsilon^2}$, the floored speed that also smooths drag and rolling resistance. The lateral forces are Pacejka's, $F_{y,w} = P_w \sin\bigl(C \arctan(B_w \alpha_w)\bigr)$, with the shape factor $C \in (1, 2)$, as the parameter's documentation requires. Before libcvc #852 the front slip was $\varsigma\delta - \operatorname{atan2}(v_y + l_f r, \bar{u})$, with $\varsigma = v_x/\bar{u}$, which left the remainder of Section 4.4.

Equations of motion. With θ the sine of the slope along the heading (positive uphill), and $c_x(\bar{u}) = c_d\bar{u} + c_{rr}mg/\bar{u}$ the drag and rolling resistance acting on v_x, the body-frame Newton–Euler equations are

$$\begin{aligned} m(\dot{v}_x - r v_y) &= F_{x,f}\cos\delta - F_{y,f}\sin\delta + F_{x,r} - c_x v_x - mg\theta, \\ m(\dot{v}_y + r v_x) &= F_{x,f}\sin\delta + F_{y,f}\cos\delta + F_{y,r}, \\ I_z\dot{r} &= l_f\bigl(F_{x,f}\sin\delta + F_{y,f}\cos\delta\bigr) - l_r F_{y,r}. \end{aligned} \tag{6}$$

In momenta and the directions above, (6) reads

$$\dot{p} = \mathsf{J}(p)v + t_f F_{x,f} + t_r F_{x,r} + n_f F_{y,f} + n_r F_{y,r} - c_x v_x\, e_x - mg\theta\, e_x. \tag{7}$$

Left out on purpose: suspension, lateral load transfer and contact loss; the four-wheel model (Section 6) adds some of these. Wheel spin and the slope across the heading are opt-in (Section 7).

4 The port-Hamiltonian form

4.1 The tires' secant stiffness

The key to an exact form is to write each lateral force as a stiffness times a velocity, in a way that stays valid past the tire's peak. A tangent stiffness does not. The secant through the force's root does.

Lemma 1 (Secant stiffness)**.** *With the slips* (5)*, $B_w > 0$, $C \in (0, 2)$ and $P_w \geq 0$, the secant stiffness*

$$S_w = -\frac{F_{y,w}}{w_w} \qquad (w_w \neq 0), \tag{8}$$

continued at the root by the slope there, $S_w = P_w C B_w u_w/(u_w^2 + w_w^2)$, is non-negative at every slip, every speed and every steering angle, sliding included. So $F_{y,w} = -S_w\, w_w = -S_w\, n_w^\top v$.

Proof. Since $u_w \geq \varepsilon > 0$, $\alpha_w = -\operatorname{atan2}(w_w, u_w)$ has the sign of $-w_w$ and vanishes exactly at $w_w = 0$: each slip's root is where its contact patch has no sideways velocity in its wheel's frame. Since $C \arctan(B_w \alpha_w) \in (-C\pi/2,\, C\pi/2) \subset (-\pi, \pi)$, the force $P_w \sin(C \arctan(B_w \alpha_w))$ has the sign of α_w, or is zero. Hence $F_{y,w}$ and w_w have opposite signs, and $S_w \geq 0$. At the root, $-\partial F_{y,w}/\partial w_w$ (at fixed u_w) is $P_w C B_w u_w/(u_w^2 + w_w^2) \geq 0$. □

The implementation is `dyn_secant_w`. It takes the root branch when $(B_w \alpha_w)^2 < 10^{-8}$, so there $F_{y,w} = -S_w w_w$ holds to second order in the slip, not exactly. That rounding is the only remainder; it is booked on its own line (Section 4.4). A Pacejka force with $C < 2$ never changes sign against its slip. A tire past its peak gives less force for more slip, but never pushes the slip further.

4.2 The structure

Define

$$\mathsf{R}(v; \delta, a) = c_x(\bar{u})\, e_x e_x^\top + S_f\, n_f n_f^\top + S_r\, n_r n_r^\top, \tag{9}$$

$$\mathsf{G}(\delta) = \begin{bmatrix} t_f & t_r \end{bmatrix}, \qquad u = (F_{x,f},\, F_{x,r}), \qquad y = \mathsf{G}(\delta)^\top v, \tag{10}$$

$$f_\varepsilon = n_f (F_{y,f} + S_f\, n_f^\top v) + n_r (F_{y,r} + S_r\, n_r^\top v). \tag{11}$$

R depends on the actuator state a through the grips P_w in S_w. f_ε is what the secant dissipation leaves of the tires' actual forces. By Lemma 1 it is zero off the secants' root branch, and that branch's rounding on it (Section 4.4). In code these are `dyn_ph_J`, `dyn_ph_R`, `dyn_ph_G`, `dyn_ph_input` and `dyn_ph_floor`. Their sum is `dyn_ph_rhs`, and `dyn_ph_rates_at` evaluates the power balance at a state.

Proposition 1 (The dynamic single-track vehicle is port-Hamiltonian at every slip)**.** *Under the conditions of Lemma 1, with $c_d, c_{rr} \geq 0$, the vehicle's equations of motion* (6)*, with its pose's kinematics, are exactly the pH system on $SE(2) \times \mathfrak{se}(2)^*$*

$$\begin{pmatrix} \dot{q} \\ \dot{p} \end{pmatrix} = \left[\begin{pmatrix} 0 & T(q) \\ -T(q)^\top & \mathsf{J}(p) \end{pmatrix} - \begin{pmatrix} 0 & 0 \\ 0 & \mathsf{R}(v; \delta, a) \end{pmatrix} \right] \begin{pmatrix} \nabla_q H \\ \nabla_p H \end{pmatrix} + \begin{pmatrix} 0 \\ \mathsf{G}(\delta)\, u + f_\varepsilon + f_g \end{pmatrix}, \qquad y = \mathsf{G}(\delta)^\top v, \tag{12}$$

with $q = (x, y, \psi)$ the pose, $T(q)$ its velocity map (below), $H = \frac{1}{2} p^\top M^{-1} p$, and $f_g = -mg\theta\, e_x$ the grade's force. The structure matrix is skew and satisfies the Jacobi identity, and $\mathsf{R} = \mathsf{R}^\top \succeq 0$ at every

state, slip and input. Since H does not depend on q, $\nabla_q H = 0$: the first row is the reconstruction $\dot{q} = T(q)v$, and the second is

$$\dot{p} = (\mathsf{J}(p) - \mathsf{R}(v;\delta,a))v + \mathsf{G}(\delta)\,u + f_\varepsilon - mg\theta\,e_x. \tag{13}$$

Consequently

$$\dot{H} = -v^\top \mathsf{R} v + y^\top u + v^\top f_\varepsilon - mg\theta\,v_x, \tag{14}$$

and the vehicle is passive with respect to the supply $y^\top u + v^\top f_\varepsilon - mg\theta v_x$.

Proof. The first row is the pose's kinematics, since $\nabla_p H = v$ and $\nabla_q H = 0$. For the second, by (11), $n_f F_{y,f} + n_r F_{y,r} = -(S_f n_f n_f^\top + S_r n_r n_r^\top)v + f_\varepsilon$. Substituting this into (7), with $t_f F_{x,f} + t_r F_{x,r} = \mathsf{G}u$ and $c_x v_x\, e_x = c_x\, e_x e_x^\top v$, gives (13). The structure matrix is the canonical structure of $T^*SE(2)$ (below), so it is skew and satisfies the Jacobi identity; its lower block $\mathsf{J}(p)$ is the Lie–Poisson tensor (2). R is a combination of outer products with coefficients $c_x \geq 0$, $S_f \geq 0$ and $S_r \geq 0$ (Lemma 1), so it is symmetric positive semi-definite. Finally $\dot{H} = \nabla_q H^\top \dot{q} + v^\top \dot{p} = v^\top \dot{p}$, $v^\top \mathsf{J} v = 0$ and $v^\top \mathsf{G} u = y^\top u$. □

The pose and $T(q)$. $SE(2)\times\mathfrak{se}(2)^*$ is the cotangent bundle $T^*SE(2)$, trivialized by left translation, and the block matrix in (12) is its canonical structure [6]. The off-diagonal blocks pair the pose with its momenta, and the lower block is the Lie–Poisson structure of $\mathfrak{se}(2)^*$. For the pose of the centre of gravity, $T(q) = R(\psi)\oplus 1$, so $\dot{q} = T(q)v$ is $(\dot{x},\dot{y}) = R(\psi)(v_x, v_y)$, $\dot{\psi} = r$. The plant's pose is the rear axle's instead (`dyn_bike.h`), as `bike_step`'s is. That pose is the centre of gravity's moved l_r back along the body's axis, a fixed right translation on $SE(2)$, so its block is

$$T(q) = \begin{pmatrix} \cos\psi & -\sin\psi & l_r\sin\psi \\ \sin\psi & \cos\psi & -l_r\cos\psi \\ 0 & 0 & 1 \end{pmatrix},$$

which gives $\dot{x} = \cos\psi\, v_x - \sin\psi\,(v_y - l_r r)$, $\dot{y} = \sin\psi\, v_x + \cos\psi\,(v_y - l_r r)$ and $\dot{\psi} = r$: the rear axle's velocity, as the step moves it (Section 5). The two are one system. The change of pose is a diffeomorphism of the phase space that leaves p alone, and it carries the canonical structure to the one with the rear axle's $T(q)$, which is therefore skew and satisfies the Jacobi identity too. We use the rear axle's, since it is the plant's state. The pose's kinematics is checked through `DynBike.Golden`, whose outputs include the pose, in the kinematic limit through `DynBike.KinematicLimitFollowsBikeStep` (Table 3), and, since libcvc #987, on its own: `DynPHFull.ThePoseUpdateIsTv` finds the step's pose update to be $q_1 = q_0 + h\,T(q_1)v_1$ to round-off (Table 10).

Corollary 1 (Level ground: the reduced system)**.** *On level ground ($\theta \equiv 0$), H and every force in* (13) *are independent of q, so the momentum equation closes on $\mathfrak{se}(2)^*$. It is then a pH system on $\mathfrak{se}(2)^*$ alone, with the Lie–Poisson structure $\mathsf{J}(p)$, and the pose follows from its solution by the reconstruction $\dot{q} = T(q)v$. Since f_ε is zero but for the secants' rounding at the root, the vehicle is then passive from the drive port (u, y) alone, at every speed and every floor ε.*

With a grade, θ is read from the world at the vehicle's position, so the momentum equation is coupled to the pose through that input (Section 4.5). With the side slope's option, gravity is in H instead (Section 7.2).

4.3 Steering is not a power port

In (13) the steering angle appears only inside n_f and t_f, and through them in the front slip and its secant. It turns the directions along which the front tire pushes, so it modulates the structure. It is not an effort with a conjugate flow, and it does no work on the body. The power that drives the body is $y^\top u = F_{x,f}\, t_f^\top v + F_{x,r}\, t_r^\top v$: each axle's force times its wheel's ground speed. With no axle force, nothing is supplied, however the wheel turns.

This matters because the natural linear-tire pH form gets it wrong, and libcvc's own first pH form (transfix/libcvc#748, replaced in #831) did. With linear tires $F_{y,f} \approx C_f \alpha_f$ and small angles, it is tempting to take $u = (F_x, \delta)$, with input column $C_f(0, 1, l_f)^\top$ for δ. The conjugate output of δ is then $C_f(v_y + l_f r)$, and the supplied power contains $C_f \delta (v_y + l_f r)$. This has no sign, so that model can gain kinetic energy from steering alone, with no drive force. It is the first-order truncation in δ of (9). Expanding $n_f \approx (-\delta, 1, l_f)$ and, at small slip, $S_f \approx C_f/u_f \approx C_f/\bar{u}$, the v_x-column of $\frac{C_f}{\bar{u}} n_f n_f^\top$ contributes $+C_f \delta\, (v_x/\bar{u})\, (0, 1, l_f)^\top$ to $-\mathsf{R}v$, which is $C_f \delta\, (0, 1, l_f)^\top$ when $v_x \approx \bar{u}$. The truncation moves that term out of R and calls it an input, and it drops the cornering drag $-F_{y,f} \sin\delta$ on v_x. In the modulated form, the front tire's power $F_{y,f}\, n_f^\top v$ is the dissipation $-S_f (n_f^\top v)^2 \le 0$, exactly but for the root branch's rounding. Steering only redirects energy.

4.4 The speed floor

The floor ε enters the slips (5) only through their forward speeds u_w, which keep them smooth through zero speed. It does not move their roots: each slip vanishes where $n_w^\top v = 0$, which is where the secant form puts it. So off the secants' root branch $f_\varepsilon = 0$ exactly, and on it f_ε is that branch's rounding. The floor line of the books is kept, and measures it. The test `DynBikePH.TheFloorTermIsRoundOff` finds its worst share of the flow at most 2.2×10^{-16} at every floor from $\varepsilon = 0.5$ down to 10^{-9} m/s.

This holds since libcvc #852, which moved the front slip into its wheel's frame. Before it, the front slip $\varsigma\delta - \operatorname{atan2}(v_y + l_f r, \bar{u})$, with $\varsigma = v_x/\bar{u}$, had its root at $v_y + l_f r = \bar{u} \tan(\varsigma\delta)$, not where $n_f^\top v = 0$ ($v_y + l_f r = v_x \tan\delta$). The remainder was $f_\varepsilon = S_f'\big(\bar{u} \tan(\varsigma\delta) - v_x \tan\delta\big)\, n_f$, with S_f' that slip's secant in $v_y + l_f r$, and R's front term was $(S_f'/\cos\delta)\, n_f n_f^\top$, positive semi-definite only for $|\delta| < \pi/2$. The bracket is $O(\varepsilon^2/|v_x|)$, and the floor's worst share of the flow fell as ε^2: 5.4×10^{-4}, 5.4×10^{-6} and 5.4×10^{-8} at $\varepsilon = 0.5$, 0.05, 0.005 m/s. Outside a crawl the change moves the plant little: the vector field, the books' order and the kinematic limit of Table 3 measure the same before and after it.

4.5 Grade

On a slope, gravity along the heading does work $-mg\theta\, v_x$. In the plant the grade is data: `dyn_plant` takes θ, the sine of the slope along the heading (positive uphill), sampled from the world each step, and does not differentiate it. So in (12) it is an outside, time-varying input $\theta(t)$, with the force $f_g = -mg\theta\, e_x$ and its own power line $-mg\theta\, v_x$. It is not the gradient of a potential, because by default the plant has no side slope. In the centre of gravity's pose, gravity on a terrain $z(x, y)$ has the in-plane force $-T(q)^\top \nabla_q (mgz) = -mg\big(R(\psi)^\top \nabla z,\, 0\big)$. That force has a component along the heading and one across it, and the plant keeps only the first. So $-mg\theta\, v_x = -\frac{d}{dt}(mgz)$ only where v_y or the side slope is zero (and to $O(\text{slope}^3)$, below).

libcvc #987 adds the side slope as an option. With it, gravity moves into H as $V(q) = mg\, z(q)$, and the grade line becomes $-\Delta V$ (Section 7.2). That holds for the potential in projected coordinates, whose force along the heading is mg times the slope's tangent. The plant's θ is its sine, and the two

agree to $O(\text{slope}^3)$; the side slope's option uses the tangents. The four-wheel plant already puts its terrain in H, by option, through its suspension (Section 6).

4.6 Tangent versus secant

Past a tire's peak, $|B_w \alpha_w| > \tan(\pi/(2C))$, its force falls as its slip grows. The *tangent* dissipation, minus the derivative of the tires' lateral force Q_y and yaw moment Q_r in (v_y, r), is

$$D_t = -\frac{\partial(Q_y, Q_r)}{\partial(v_y, r)} = k_f\, a a^\top + k_r\, b b^\top, \qquad a = (1, l_f),\ b = (1, -l_r), \tag{15}$$

with k_w each tire's $\partial F_{y,w}/\partial \alpha_w$ times its slip's sensitivity to (v_y, r) along a or b. At the rear that is $u_r/(u_r^2 + w_r^2)$. At the front, the sideways velocity w_f and the forward speed $t_f^\top v$ (floored to u_f) both move along a, as $\cos\delta$ and $\sin\delta$, and the force reaches the body through $\cos\delta$, so

$$k_f = \frac{\partial F_{y,f}}{\partial \alpha_f}\, \frac{\cos\delta\,(\cos\delta\, u_f - \sin\delta\, w_f\, t_f^\top v/u_f)}{u_f^2 + w_f^2}. \tag{16}$$

In code this is `dyn_ph_tangent_D`; `DynBikePH.TheTangentIsTheTiresDerivative` checks it against central differences of the tires' lateral force and moment, to 1.4×10^{-7} of its scale. Since a and b are independent, $D_t \succeq 0$ exactly when both $k_w \geq 0$: when both tires are below their peak (and the front's slip angle is under about $90° - |\delta|$, where its sensitivity changes sign). Past the peak, the linearized motion can be locally unstable, which is sliding. That is a statement about stability, not about energy. The secant (9), the dissipation in the energy balance (14), stays positive semi-definite. So a positive semi-definite dissipation of the form $-D(v)v$ can represent sliding: its gain falls with slip. What it cannot represent is a tire that supplies energy, and a Pacejka tire with $C < 2$ never does.

On the tests' full-size utility-vehicle preset (`DynBikePH.TheSecantStaysPSDWhereTheTangentDoesNot`), with steer ramps to the lock and swerves at 8–20 m/s:

- The vehicle understeers: the front tire passes its peak first.
- The tangent is positive semi-definite in every sample below about half the grip, $0.5\mu g$. Above it, it is indefinite in up to half the samples of a lateral-g bin.
- The rear tire never passes its peak.
- The secant is positive semi-definite in every sample, at $\mu = 0.9$ and at $\mu = 0.6$.

This bears on any learned model that constrains $D(v) \succ 0$, such as the one in Section 8. The constraint does not rule out sliding.

4.7 The kinematic limit and the Jacobi identity

The kinematic bicycle imposes a no-slip constraint at each axle. That constraint is not integrable, so the reduced bracket is only almost-Poisson and the Jacobi identity fails [8]. The dynamic vehicle replaces the constraint by tire forces, its J is a genuine Lie–Poisson structure, and the kinematic bicycle is its stiff limit: $S_w \to \infty$ along n_f and n_r drives $n_w^\top v$ to zero. The test `DynBike.KinematicLimitFollowsBikeStep` checks the limit numerically. It uses stiff tires and puts the mass at the rear axle, as the kinematic bicycle has it. Under the kinematic bicycle's commands, the dynamic vehicle then stays within 0.011 (normalized units) of the kinematic path over 200 ticks, with a mean path length of 5.6.

4.8 Externally defined ports

The form (13) accepts further ports. Suppose an external generalized force acts on the body: a tow, wind, a contact, or a force defined by some other model. It enters through its own input map, $\mathsf{G}_e u_e$, and has its own conjugate output $y_e = \mathsf{G}_e^\top v$. The balance gains one line, $y_e^\top u_e$, which is booked as supplied power:

$$\dot{p} = (\mathsf{J} - \mathsf{R})v + \mathsf{G}u + \mathsf{G}_e u_e + f_\varepsilon - mg\theta e_x, \qquad \dot{H} = -v^\top \mathsf{R} v + y^\top u + y_e^\top u_e + v^\top f_\varepsilon - mg\theta v_x. \quad (17)$$

Nothing else changes: J and R are untouched, and passivity holds with the added supply. For a force $F = (F_X, F_Y)$ given in the world frame and applied at the centre of gravity, the body wrench is $(R(\psi)^\top F, 0)$. So

$$\mathsf{G}_e = \begin{pmatrix} R(\psi)^\top \\ 0 \quad 0 \end{pmatrix}, \qquad y_e = R(\psi) \begin{pmatrix} v_x \\ v_y \end{pmatrix},$$

and y_e is the centre of gravity's world velocity. The supplied power is $F \cdot \dot{x}_{\mathrm{cg}}$, as it should be. Applied at a body point (ℓ_x, ℓ_y) instead, the yaw row of G_e becomes $(-\ell_y, \ell_x)R(\psi)^\top$, the moment arm. A port that stores energy, such as a spring, belongs in H instead.

Worked example: a navigation force. GRL-SNAM and libcvc allow an external navigation force F_{ext}, with its spatial Jacobian. It might come from another field, a tow or a learned term. It is joined to the navigator's conservative force (`ext_in` in `bike_control`; `grl_snam.ext_rollout`, `material_ext`), and it reaches the two models differently.

- On the navigator's point-mass surrogate, F_{ext} is a port force on the point mass. The per-step books (Section 9) carry it as the port work $W_{\mathrm{port}} = \frac{h}{2} F_{\mathrm{ext}} \cdot (v + v')$. Left out, it is work of order h, against a bound of order h^2. The test `NavPotential.UnloggedForceRaisesTheAlarm` sees exactly that: 229 of 240 steps raise the alarm.
- On the vehicle, F_{ext} acts through the controller. It changes $(\delta_{\mathrm{cmd}}, a_{\mathrm{cmd}})$, so it reaches the body only through the axle forces, which are already booked as $y^\top u$, and through steering, which does no work. The plant's books need no new line: the external force's energy shows up as drive or brake. A model in which the force acts on the body directly, as a physical force, would add the column G_e above, with F_{ext} as u_e. libcvc does not implement such a column for F_{ext}. Compliant contact (Section 7.3) is one, applied at each point of the vehicle's outline.

5 The discrete step and its energy books

5.1 The step

One step of length h (`dyn_plant`) from the commands $(\delta_{\mathrm{cmd}}, a_{\mathrm{cmd}})$ and the grade has these parts:

1. *Actuators.* The steering follows its command with an exact first-order lag, $1 - e^{-h/\tau}$, with its change limited smoothly to a slew rate times h. The force follows its command with its own lag. Their outputs (δ_1, a_1) are the inputs the rest of the step applies.
2. *Loads and axle forces* from a_1, as in Section 3.
3. *Speed, predicted* from the axle forces, drag, rolling resistance and grade alone. Lateral forces are not read before step 4: in the stiff limit they are constraint forces.

4. *Lateral and yaw, implicit.* (v_y, r) advance by implicit Euler at the predicted speed. The step is solved by a few Newton-like iterations, each linearizing the tires by their secants at its iterate. A sliding tire then pulls its slip toward zero, never past it, and stiff tires stay stable at the navigator's 0.06 s tick.

5. *Speed, corrected* with the front lateral force the implicit step applied, then *clamped* to the vehicle's speed range.

6. *Low-speed blend.* Below a blend speed, a sigmoid in $|v_x|$ blends (v_y, r) toward the kinematic bicycle's, where slip angles mean nothing.

Then the pose advances by the reconstruction $\dot{q} = T(q)v$ of Proposition 1, at the new heading and twist: $\psi' = \psi + hr'$, and the rear axle moves by h times its velocity $(v_x', v_y' - l_r r')$ turned by ψ', so $q' = q + h\,T(q')v'$. The pose does not enter H, unless the side slope's option puts gravity there (Section 7.2). The step is first order in h.

5.2 The books

`dyn_plant_books` runs exactly that step and returns, in the struct `dyn_ph_books`,

$$\Delta H = \underbrace{\widehat{\int} y^\top u}_{\text{supplied}} - \underbrace{\widehat{\int} v^\top \mathsf{R} v}_{\text{dissipated}} + \underbrace{\widehat{\int} v^\top f_\varepsilon}_{\text{floor}} \underbrace{- \widehat{\int} mg\theta v_x}_{\text{grade}} + W_{\text{lim}} + r_h. \tag{18}$$

Each $\widehat{\int}$ is the trapezoid, over h, of that power. Its two ends are the step's start (the start twist, with the inputs (δ_1, a_1) the step applied) and the twist the step reached *before* its limiters, the speed clamp and the low-speed blend. W_{lim} is the kinetic energy those limiters then changed, booked exactly as the difference of H across them. r_h is what is left, the integrator's residual. The step's arithmetic does not depend on the books: `dyn_plant` records the pre-limiter twist only when asked, and the books left `DynBike.Golden`'s outputs unchanged.

Proposition 2 (The books close to second order)**.** *Where the vector field is smooth along the step, $r_h = O(h^2)$ per step, so over a fixed horizon T the summed residual is $O(h)$.*

Argument. Before its limiters, the step is a consistent first-order method: implicit Euler on (v_y, r), and predicted-then-corrected Euler on v_x with the same forces. So the pre-limiter twist differs from the exact flow's twist at $t + h$ by $O(h^2)$. Along the exact flow, $H(t+h) - H(t) = \int_t^{t+h} \dot{H}$ by (14), and the trapezoid of a smooth integrand has error $O(h^3)$. Replacing the exact end state by the step's changes H by $O(h^2)$ and the trapezoid by $h \cdot O(h^2)$. The limiters' energy is booked exactly. Hence $r_h = O(h^2)$. The finite number of Newton iterations, and the branch switches (load floor, saturation, the secant's root branch), are part of r_h, not exceptions to it. □

`DynBikePH.StepBooksCloseToSecondOrder` checks this on a swerve past the tire's peak, with the throttle working and the actuators lagging, at $\mu = 0.6$. As h halves from 0.02 to 0.0025 s, the mean per-step $|r_h|$ falls by 3.83, 3.92 and 3.95. Over 6 s at $h = 0.01$ s the summed residual is 1.2×10^{-3} of the energy that moved (the flow: the sum of the magnitudes of every line). Braking hard to a stop in a turn makes the clamp and the blend act. The books charge them (for example -358 J at $h = 0.01$ s), and the rest closes to 4×10^{-6} of the flow.

5.3 Closing the books exactly

`diff::dyn_plant_dg` (`inc/cvc/nav/detail/dyn_bike_dg.h`, libcvc #978) is an opt-in step whose books close exactly: the discrete gradient of (13). $H = \frac{1}{2} v^\top M v$ is quadratic in the twist, so its discrete gradient is the midpoint twist, and the step is the implicit midpoint rule

$$M \frac{v_1 - v}{h} = \mathsf{J}(M\bar{v})\,\bar{v} + Q(\bar{v}), \qquad \bar{v} = \tfrac{1}{2}(v + v_1), \tag{19}$$

with Q the tires, the axle forces, drag, rolling resistance and grade, taken at the midpoint state with the actuators' outputs at the step's midpoint. Since J is skew, $\Delta H = h\,\bar{v}^\top Q(\bar{v})$ holds line for line, and `dyn_plant_dg_books` closes to the tolerance of the Newton solve. The actuators, the speed clamp and the low-speed blend are `dyn_plant`'s, and the books charge the limiters as (18) does. `dyn_plant` and its callers are untouched, so the default is unchanged.

- On the swerve of Proposition 2's test, from $h = 0.06$ to 0.0025 s, the residual is at most 1.7×10^{-10} J a step, at most 1.3×10^{-12} of the step's flow, where `dyn_plant` leaves 25 J to 0.056 J a step.
- It is second order: its error at 2 s falls 3.92 to 4.03 times per halving of h, where `dyn_plant`'s falls 1.98 to 2.13 times.
- No coasting step gains energy, at the navigator's 0.06 s step or at the live plant's six 0.01 s substeps, with tires up to sixteen times stiffer than the vehicle's.
- It is A-stable but not L-stable. A tire much stiffer than the step can resolve swings its slip past zero and back, decaying, where `dyn_plant`'s implicit Euler lands on the root in one step. At the live plant's substeps with the vehicle's own tires it swings no more than `dyn_plant` does.
- A step costs 2.8 to 3.1 μs in double, about ten times `dyn_plant`'s 0.30 μs.

It covers the twist without wheel spin, and it is C++ only: `pycvc` does not bind it, and GRL-SNAM has no twin of it. Table 4 gives the gates.

With the wheels spinning. `diff::dyn_spin_plant_dg` (libcvc #1045) extends the step to the wheel-spin plant of Section 7.1. Its H (26) is quadratic in $e = \nabla H = (v, \omega)$, and Q is that plant's own generalized force (`dyn_spin_ph_forces_at`). The body's rows take the midpoint, as in (19). The wheels' slip is far stiffer than the step: its rate times the navigator's 0.06 s step is 60 to 600. At the midpoint, which is A-stable but not L-stable, a disturbed slip, such as rims spun 1.5 times the ground's speed, swings every step, and five substeps still swing 98, 36 and 8 times in 50 steps coasting from 0.5, 3 and 12 m/s. So the wheels' rows take the step's end:

$$\mathcal{M} \frac{e_1 - e}{h} = \mathsf{J}(\mathcal{M}\tilde{e})\,\tilde{e} + Q(\tilde{e}), \qquad \tilde{e} = e + \Theta(e_1 - e), \qquad \Theta = \mathrm{diag}(\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}, 1, 1), \tag{20}$$

with $\mathcal{M} = \mathrm{blockdiag}(M, J_f, J_r)$. Since J is skew,

$$\Delta H = h\,\tilde{e}^\top Q(\tilde{e}) - \tfrac{1}{2}(J_f \Delta\omega_f^2 + J_r \Delta\omega_r^2)$$

exactly. The second term is the scheme's own damping, and `dyn_spin_plant_dg_books` carries it on a line of its own, `scheme`, so the books still close to the solve's tolerance: at most 1.3×10^{-12} of the step's flow, where `dyn_spin_plant` leaves 20 to 240 J a step at 0.06 s. The scheme's line is small

on a swerve (21 J at 0.06 s) but not on a launch that spins the wheels, where it takes 4.2×10^4 J against the slip's 4.4×10^5 J. The wheels' speeds are first order, but at every step size the step is 10 to 13 times nearer its converged solution than `dyn_spin_plant` is. Its slip swings about as often as `dyn_spin_plant`'s, and no coasting step gains energy. A step costs 11 to 16 μs at 0.06 s, 19 to 27 times a `dyn_spin_plant` step. Like `dyn_plant_dg`, it is opt-in and C++ only. Table 5 gives the gates.

5.4 The live plant

`cvc::physics::dyn_bike_plant` (libcvc #833) makes the same vehicle a plant that a navigation world (`cvc::nav::sim_world`) drives. It sits beside the kinematic bicycle and the rigid-body engine's vehicles, and needs no physics world. Agent i drives one of a list of vehicle descriptions, with parameters from `nav_plant::dyn_bike_si_of`, so the dynamic and rigid-body plants read one description. Each tick of length Δt runs $\lceil \Delta t/h \rceil$ steps under the held command, with $h = 0.01$ s by default (six steps at the navigator's 0.06 s tick), in SI units. It books every step through `dyn_plant_books`. The books are kept per tick (`last_tick`) and since the agent was attached (`books`):

$$K - K_0 = \text{drive} - \text{brake} - \text{dissipated} - \text{slip} + \text{floor} + \text{grade} + \text{limiter} + \text{contact} \\ - \text{stored}_\text{c} - \text{dissipated}_\text{c} + \text{exchange}_\text{c} + \text{residual}, \quad (21)$$

where drive and brake are the positive and negative parts of each step's supplied term, split step by step. The slip line and the three lines marked c are zero unless wheel spin or compliant contact is on (Section 7); with wheel spin, K includes the wheels' spin. It is selected through `plant_host` as `"dynamic"`, or `"dynamic:<presets>"` to cycle vehicles. It is the plant that the coefficient trainer's `rollout_kind::dynamic` trains through, so a policy trained there is evaluated on its own model.

Ground, world and driver. When Table 6's rows were measured, the plant was flat (grade zero) and had no world to collide with, and the contact line was zero. libcvc #869 and #873 add three things:

- *Ground.* Given a ground height (`plant.ground_height`), the step takes θ as the sine of the slope along the heading under the centre of gravity, over a wheelbase, and the grade line books it (Section 4.5).
- *World.* Given a rigid-body world (`dyn_bike_plant_options::physics`), each chassis is a box in it, moved every substep as its model moved. A box that touches anything but the ground takes the world's pose and twist back into its model, and the energy that changes is booked as contact. This contact is outside the pH form: it replaces the state rather than entering through H, R or a port (Section 12.1); compliant contact (Section 7.3) is the alternative inside it. With Jolt in the build, `plant_host` now builds that world for `"dynamic"`, holding the scene's ground and buildings.
- *Driver.* The navigator's acceleration goes through the rigid-body plant's driver, on by default (`dyn_bike_plant_options::driver`). It holds the vehicle on its brakes at a standstill and corrects the drive force for the slope and for what the last substep fell short by.

The four-wheel vehicle is a live plant in the same way, with its grade force on: libcvc #869's `dyn_4w_plant`, which `plant_host` selects as `"dynamic4w"`. Table 7 gives the tests of these additions. Three more options of the single track's live plant are off by default (Section 7): `wheel_spin`; `side_slope`, which reads the same ground height; and `contact`, compliant contact against a field (`contact_field`) and between the vehicles. Compliant contact takes the world's place: a plant uses one or the other.

6 The four-wheel vehicle

libcvc's two-track vehicle, `diff::dyn4_plant` (`inc/cvc/nav/detail/dyn_4w.h`, twinned as `grl_snam.dyn4`), has four contact patches with Ackermann front wheels, a spring and a damper at each corner, and heave, roll and pitch. It leaves out the unsprung masses, camber and the anti-roll bars, and has wheel spin only by option (Section 7.1). libcvc #850 writes it in pH form at every slip, with per-step books, as Sections 4 and 5 do for the single track. The plant's arithmetic does not depend on either: `Dyn4.Golden` and `Dyn4ExtRollout.Golden` pass unchanged.

State and structure. The state is $x = (q, p)$, nine coordinates: $q = (z, \phi, \vartheta)$ is the body's heave, roll and pitch from rest, and $p = M\xi$ the momenta of its twist $\xi = (v_x, v_y, r, \dot z, \dot\phi, \dot\vartheta)$, with $M = \mathrm{diag}(m, m, I_z, m, I_x, I_y)$. The pose, the rear axle's as in the single track, follows from the twist by the same reconstruction $\dot q = T(q)\,(v_x, v_y, r)$ (Proposition 1), and enters only through the ground under the wheels. With $\nabla H = (\partial H/\partial q,\, \xi)$ and $u = (F_{x,1}, \ldots, F_{x,4})$ the wheels' drive or brake forces,

$$H = \tfrac{1}{2}\xi^\top M\xi + \textstyle\sum_i \tfrac{1}{2}k_i l_i^2 + mgz, \qquad \dot x = (\mathsf{J} - \mathsf{R})\nabla H + \mathsf{G}u + f_{\mathrm{lever}}, \qquad y = \mathsf{G}^\top \nabla H, \tag{22}$$

where l_i is corner i's spring compression; the springs' preloads hold the body at $q = 0$ on flat ground. This is the default plant, without the grade force described below.

- $\mathsf{J} = \begin{pmatrix} 0 & 0 & I \\ 0 & \mathsf{J}_h(p_h) & 0 \\ -I & 0 & 0 \end{pmatrix}$ over (q, p_h, p_v), the horizontal and vertical momenta: canonical in the vertical, and the Lie–Poisson structure (2) in the horizontal.

- $\mathsf{R} = \mathrm{blockdiag}\,(0,\ c_x e_x e_x^\top + \sum_i S_i n_i n_i^\top,\ \sum_i c_i' c_i c_i^\top)$. Here $n_i = (-\sin d_i, \cos d_i, x_i \cos d_i + y_i \sin d_i)$ carries wheel i's lateral force, at its offset (x_i, y_i) and angle d_i, onto (v_x, v_y, r). So $n_i^\top v$ is the lateral velocity w_i in that wheel's frame, where its slip is measured, and $S_i = -F_{y,i}/w_i \geq 0$ is its secant (Lemma 1). The last block is the dampers: c_i' is corner i's damping and $c_i = (1, y_i, -x_i)$ takes $\dot q$ to its vertical rate. So R is positive semi-definite at every slip and every steering angle.

- G's columns are $T_i = (\cos d_i, \sin d_i, x_i \sin d_i - y_i \cos d_i, 0, h \sin d_i, -h \cos d_i)$ on the twist: wheel i's force along the wheel, at its contact patch, h below the centre of gravity. So y_i is that patch's speed along its wheel, roll and pitch included, and $y^\top u$ is the forces' whole power. Steering is not an input: the angles d_i only turn n_i and T_i.

- $f_\varepsilon = 0$. Each slip is in its wheel's frame and rooted at $w_i = 0$, and the form takes the exact secant, so nothing is left at the speed floor.

The lever. One term fits neither J nor R:

$$f_{\mathrm{lever}} = (0,\ h\textstyle\sum_i F_{y,i} \cos d_i,\ h\sum_i F_{y,i} \sin d_i) \quad \text{on } (p_z, p_\phi, p_\vartheta),$$

the moments of the tires' lateral forces on roll and pitch. The slips do not see the body's roll and pitch rates (a patch's lateral velocity has no $h\dot\phi$ in it), so the coupling runs one way. It is neither skew nor dissipative, and it is booked on its own line, $\xi^\top f_{\mathrm{lever}}$, which vanishes with h. So

$$\dot H = -\nabla H^\top \mathsf{R} \nabla H + y^\top u + \xi^\top f_{\mathrm{lever}} + \textstyle\sum_i k_i l_i\, \dot h_{g,i}, \tag{23}$$

and the vehicle is passive with respect to its wheels' forces except for the lever. The last term is the ground under wheel i, at height $h_{g,i}$, lifting its spring as the wheel rolls over it; it is zero on flat ground. The lever is small: over 8 s of swerving past the tires' peak on the tests' full-size preset at $\mu = 0.6$, it books about 1.0 kJ against 277 kJ dissipated.

Grade. By default the plant has no grade force on its horizontal motion. The terrain's height enters only the springs, whose bases it moves, and z is heave from rest, not elevation. A vehicle then climbs a hill for free, and the books show that energy as the ground line of (23), from outside. The opt-in `dyn4_params::grade_force` adds each spring's horizontal reaction on the slope under its wheel, f_{grade}, on (v_x, v_y, r). It is minus the gradient of the springs' energy in the pose, through the ground under the wheels. With the pose in the state, $T^*SE(2)$'s canonical structure puts it in J, and the terrain's energy is in H; libcvc #987 writes that form out and checks it (Section 7.2). Its power, booked as a grade line, cancels the ground line: $\xi^\top f_{\text{grade}} = -\sum_i k_i l_i \dot{h}_{g,i}$. Coasting up a 5% ramp for 2 s from 10 m/s, with drag, rolling resistance and the dampers off, the vehicle keeps its speed without the force while H gains mg times the climb. With it, it loses 0.986 m/s, against $g\,s\,t = 0.981$, and H holds to within 0.8%. The option is off by default, so the plant's goldens and GRL-SNAM's `dyn4.py` twin are unchanged; the twin does not have it.

The books. `dyn4_plant_books` runs `dyn4_plant`'s step, to the last bit, and returns

$$\Delta H = \text{supplied} - \text{dissipated} + \text{lever} + \text{grade} + \text{ground} + W_{\text{lim}} + r_h. \tag{24}$$

As in (18), each power is the trapezoid between the step's start and its pre-limiter state, both on the ground the step stood on, and W_{lim} is the limiters' change of H. The ground line is the springs' energy that the new ground under the wheels adds at the step's end. The step holds the ground and the wheels' loads at its start; that is integrator error, so it lands in r_h. On an 8 s weave at 0.2 Hz to near full lock at $\mu = 0.6$, with a tire past its peak in about 27% of the steps, $|r_h|$ falls 3.84 to 3.98 times per halving of h from 0.02 to 0.0025 s, flat and over hills. At $h = 0.01$ s the summed residual is 5.0×10^{-4} of the flow on flat ground and 1.0×10^{-3} over hills. With the grade force on, the grade and ground lines cancel to second order: their per-step sum falls 4.0 times per halving. Table 8 gives the gates.

7 Wheel spin, the side slope and compliant contact

Three opt-in extensions answer two of the gaps raised in review (Section 12.1) and the side slope that Section 4.5 leaves out: wheel spin (libcvc #982, Section 7.1), the full (q, p) form with gravity in H (#987, Section 7.2), and compliant contact (#983, Section 7.3). Each is off by default, and with it off every plant's arithmetic is as before: the goldens of Tables 3–8 pass unchanged. CVC-Lab/GRL-SNAM #146 twins all three (Section 10).

7.1 Wheel spin

In the plants of Sections 3 and 6, each axle's drive or brake force is set within its grip, so no wheel spins and none locks. The opt-in plants `diff::dyn_spin_plant` and `diff::dyn4_spin_plant` give each axle's wheels (on four wheels, each wheel) a speed ω of its own, with a spin inertia and the rolling radius r_w.

The model. The actuator's a becomes torques, split between the axles as the force was. For $a \geq 0$ the drive's torque on axle w, $T_{d,w}$, is r_w times that axle's share of ma. For $a < 0$ the brakes' torque $T_{b,w}$ is r_w times its share of $m|a|$, and it acts against the rolling, as $-T_{b,w}\,\mathrm{sg}(\omega_w)$, with sg the sign smoothed over the speed floor at the rim. A brake stops a wheel and holds it, so a negative a never drives backward. Each tire pushes along its wheel, t_w of (4), with Pacejka's longitudinal force of its slip:

$$F_{x,w} = D_w \sin\!\big(C_x \arctan(B_x \sigma_w / u_w)\big), \qquad \sigma_w = r_w \omega_w - t_w^\top v, \tag{25}$$

where σ_w is the contact patch's slip velocity (the rim's speed less the ground's along the wheel), u_w the ground's speed along the wheel, floored as in (5), and $D_w = \mu F_{z,w}$ the axle's whole grip. The lateral force takes what grip the friction ellipse leaves, and the wheel feels $-r_w F_{x,w}$. On the tests' full-size preset the wheels' radius is 0.47 m, their inertia 12 $\mathrm{kg\,m^2}$ an axle, and the longitudinal peak, taken from the rigid-body simulator's description of the same vehicle, at slip ratio 0.080.

The form. With the wheels' momenta $L = (J_f\omega_f, J_r\omega_r)$ in the state, $x = (p, L)$ has five coordinates, and

$$\begin{gathered} H = \tfrac{1}{2} v^\top M v + \tfrac{1}{2}(J_f\omega_f^2 + J_r\omega_r^2), \qquad \nabla H = (v, \omega), \\ \dot{x} = (\mathsf{J} - \mathsf{R})\nabla H + \mathsf{G}u + f_\varepsilon - mg\theta\, e_x, \qquad u = T_d - T_b\,\mathrm{sg}(\omega), \qquad y = \mathsf{G}^\top \nabla H = \omega, \end{gathered} \tag{26}$$

with

- $\mathsf{J} = \mathrm{blockdiag}(\mathsf{J}(p), 0)$: the structure couples nothing to the wheels;
- $\mathsf{R} = \mathrm{blockdiag}(\mathsf{R}_{\mathrm{body}}, 0) + \sum_w K_w d_w d_w^\top$. Here $\mathsf{R}_{\mathrm{body}}$ is (9) with each tire's lateral grip what its longitudinal force leaves; $d_f = (-t_f, r_w, 0)$ and $d_r = (-t_r, 0, r_w)$, so that $d_w^\top \nabla H = \sigma_w$; and $K_w = F_{x,w}/\sigma_w$ is the longitudinal force's secant through its root;
- $\mathsf{G} = [\, e_4 \;\; e_5 \,]$: each axle's torque on its wheels.

A Pacejka force with $C_x < 2$ has its slip's sign, so $K_w \geq 0$ by the argument of Lemma 1, and R is positive semi-definite at every slip, rolling, spinning or locked, and at any steering angle. The port is now (wheel torque, wheel speed), and $y^\top u$ is shaft power. Along solutions

$$\dot{H} = -v^\top \mathsf{R}_{\mathrm{body}} v - \textstyle\sum_w K_w \sigma_w^2 + \omega^\top T_d - \sum_w T_{b,w}\,\omega_w\,\mathrm{sg}(\omega_w) + v^\top f_\varepsilon - mg\theta\, v_x, \tag{27}$$

in which the brakes' term is non-positive at every state. Steering still does no work. On four wheels, $x = (q, p, L)$ has 13 coordinates, H keeps the springs and mgz of (22), and t_i is G's column T_i of Section 6: the patch's speed along its wheel, roll and pitch included. So the longitudinal coupling is reciprocal and goes into R whole, with no one-way term like the lateral forces' lever.

The step and its books. The step advances $(v_x, \omega_f, \omega_r)$ by implicit Euler, with the tires and the brakes linearized by their secants through their roots. The matrix it solves is symmetric positive definite, so the solve never fails, the stiff slip stays stable at the navigator's step, and a spinning or locking wheel's slip is pulled toward zero rather than past it. `dyn_spin_plant_books` and `dyn4_spin_plant_books` book each step as in (18) and (24), with the brakes and the longitudinal slip on lines of their own:

$$\Delta H = \text{supplied} - \text{braked} - \text{dissipated} - \text{slip} + \text{floor} + \text{grade} + W_{\mathrm{lim}} + r_h$$

on the single track, plus the lever and the ground on four wheels. The live plant takes wheel spin by option (`dyn_bike_plant_options::wheel_spin`), and its books gain the slip line.

What it does. With full drive on grip 0.3 for 1 s from 5 m/s, the front wheels run at slip ratio 3.57 and the rear at 1.76. Braking at 8 m/s^2 from 20 m/s on grip 0.6 locks the front wheels (slip ratio -0.97), and the vehicle stops in 34.8 m, where `dyn_plant` stops in 33.9 m. With the wheels' inertia at 1% of the preset's, the plant stays within 2 mm of `dyn_plant` over 5 s of weaving. With the preset's wheels, a 0.8 m/s^2 drive for 2 s from 10 m/s ends at 11.5369 m/s, where the effective mass $m + \sum J/r_w^2$ predicts 11.5376. The books close to second order on a swerve past the lateral peak, and faster than that on a launch that spins the wheels to slip ratio 8.7 and then locks them, while the slip mode is stiff against h. Table 9 gives the gates.

7.2 The full form and the side slope

Proposition 1 puts the pose in the state, but its H does not depend on the pose: the grade is data, and acts along the heading only (Section 4.5). libcvc #987 adds the slope across the heading, with gravity in H. With $h_g(x, y)$ the ground's height and $c(q) = (x + l_r \cos\psi,\ y + l_r \sin\psi)$ the centre of gravity of the rear-axle pose q, let $V(q) = mg\, h_g(c(q))$ and $H = \frac{1}{2} p^\top M^{-1} p + V(q)$. The structure of (12) is unchanged, and its rows become

$$\dot{q} = T(q)v, \qquad \dot{p} = -T(q)^\top \nabla_q V + (\mathsf{J}(p) - \mathsf{R})v + \mathsf{G}u + f_\varepsilon, \tag{28}$$

with $-T(q)^\top \nabla_q V = -mg(\theta_\parallel, \theta_\perp, 0)$, where $\theta_\parallel$ and $\theta_\perp$ are the ground's gradient at the centre of gravity along and across the heading. Gravity moves from the supply into H, and its power, $-\dot{V} = -mg(\theta_\parallel v_x + \theta_\perp v_y)$, is the grade line. In code these are `dyn_ph_T`, `dyn_ph_J_full` and `dyn_ph_rhs_full`, with the ground overloads `dyn_ph_potential` and `dyn_ph_grad_q`.

Tangents, not sines. $\theta_\parallel$ and $\theta_\perp$ are the gradient's components, the slope's tangents. The model's pose and twist are the vehicle's projections on the plane, so the force whose work is $-\Delta V$ is $-mg\nabla h_g$. The sine, which the momentum-only form takes as its θ, leaves $mg(\tan - \sin) \approx mg\theta^3/2$ unbooked, and a smaller step does not remove it. At a 10% grade the two differ by 0.5%.

The plant. `dyn_plant` takes an optional slope across the heading, whose force $-mg\theta_\perp$ joins the lateral solve. `dyn_plant_ground` samples both slopes from a ground under the centre of gravity, and `dyn_plant_books_ground` books the grade line as $-\Delta V$ over the step, so the kinetic ΔH less the grade line is $\Delta(T + V)$. The wheel-spin plant has the same (`dyn_spin_plant_ground`), and the live plant takes it by option (`dyn_bike_plant_options::side_slope`), reading the ground height of Section 5.4. The step's pose update is $q_1 = q_0 + h\, T(q_1)v_1$ to round-off: the reconstruction of Proposition 1, taken at the step's end.

What it shows. Closed loop across a 15% hillside with swells, with the body slipping up to 0.052 rad, for 6 s: with V in H the books close to second order, the residual falling 4.0 times per halving of h, and their sum is 2.8×10^{-5} of the flow at $h = 2.5$ ms. On the same steps, the momentum-only books leave 1.5–1.6 $\times 10^{-2}$ of the flow unbooked at every h, the plant without the side slope 1.2–1.5 $\times 10^{-3}$, and the sines 1.8–2.1 $\times 10^{-3}$: model gaps, not integrator error. The live plant, driven straight across a 10% side slope for 6 s, drifts 0.196 m downhill, and its grade line equals $-mg\Delta z$ to round-off.

Four wheels. The four-wheel plant has no side-slope gap: its grade force acts along and across the heading, and the terrain's energy is in H through the springs (Section 6). What it lacked was the pose in the state. `dyn4_ph_rhs_full` adds it, for twelve coordinates in all, and with it $-T^\top \partial H/\partial g$,

g the pose, is the grade force to round-off. So the grade force moves into the structure, as Section 6 says it does. Without the grade force, the plant's momentum rates on the tests' hillside miss up to 0.20 of the full form's. Table 10 gives the gates.

7.3 Compliant contact

The live plant's rigid-body world (Section 5.4) resolves a collision by replacing the model's state with the world's. libcvc #983 adds contact inside the single track's form, as an option of the live plant (`dyn_bike_plant_options::contact`) in place of that world, against the scene and between vehicles (`inc/cvc/nav/detail/dyn_contact.h`).

The model. A vehicle's outline about its centre of gravity, its chassis box and wheels, is sampled at points l_k: by default seven along each side and four along each end. Let ϕ be the scene's signed distance, positive in free space, read from the navigation world's own grid (`dyn_bike_plant::sdf_field`) or from any sampler, and $d_k = \phi(P_k)$ at point k's world position P_k. Within a skin s of an obstacle, each point stores a barrier energy $\psi_k = \frac{1}{2}k(s - d_k)^2$. It damps its normal velocity with the Hunt–Crossley coefficient $c_k = c\,(s - d_k)/s$, which is zero at first touch, so a point that leaves is never held back. And it slides along the obstacle with Coulomb friction on the barrier's push, its direction smoothed. Each point's world force F_k reaches the body through the external port of Section 4.8 at its arm: $\mathsf{G}_{e,k}^{\top} F_k = (R(\psi)^{\top} F_k,\ l_k \times R(\psi)^{\top} F_k)$.

The form. With the pose g of the centre of gravity in the state, as in Section 7.2,

$$H = \tfrac{1}{2} v^{\top} M v + \Phi(g), \qquad \Phi = \textstyle\sum_k \psi_k, \qquad \mathsf{R}_c = \textstyle\sum_k \left(c_k\, b_k b_k^{\top} + f_k\, a_k a_k^{\top}\right), \tag{29}$$

with $b_k = \mathsf{G}_{e,k}^{\top} \nabla \phi_k$ the point's normal and $a_k = \mathsf{G}_{e,k}^{\top} t_k$ its tangent on the twist, and f_k the friction's secant through zero sliding. The barrier's force, $-T_c^{\top} \nabla_g \Phi$ with $T_c = R(\psi) \oplus 1$, comes through the canonical structure, as gravity's does in (28). The damping and friction are $-\mathsf{R}_c v$, and R_c is positive semi-definite. So contact stores energy in H and dissipates it in R; it supplies none.

The plant and its books. The plant applies the contact's wrench as the external port, held over each substep, and books its power $y_e^{\top} u_e$. The contact's books split that into the barrier's change and the damper's work. The barrier is held from the substep's start while the step moves the pose with its new twist, so its work differs from its change by second order per step. The damper and the friction are implicit (libcvc #1004, `diff::dyn_contact_implicit`): the step solves $(M + h\mathsf{R}_c)\, v_d = M v_0 + h W_c$, with $M = \mathrm{diag}(m, m, I_z)$ and W_c the barrier's wrench, and holds $W_d = -\mathsf{R}_c v_d$, taking the friction's secant again at the sliding speed v_d reaches. So $W_d^{\top} v_d \le 0$, and without the barrier the step never raises the kinetic energy, at any h. Touching vehicles are solved together, so their forces stay equal and opposite. A substep in which a point would close more than `contact_step_depth` (0.1 m by default) on what it meets is cut into equal pieces, until nothing touches or closes. The books then close to second order through fast impacts at the default 0.01 s substep.

Before #1004 the damper and friction were held from the substep's start. Their coefficients grow with the penetration, so past some depth $h\mathsf{R}_c/m$ exceeded 2 and they reversed the approach within a step. Driven by the navigator into a wall at 7.8 m/s at 0.01 s, the full-size preset then rebounded at 3.2 m/s, the books showed the contact supplying 1.5×10^5 J, and 3.1×10^{-2} of the flow went unbooked. With #1004 the same impact dissipates 8.16×10^4 J in the contact, against 8.19×10^4 J at a 0.00125 s substep. The speed clamp takes nothing, the slowest speed is -0.47 m/s,

as at the fine step, and the books leave 1.3×10^{-3} of the flow. The navigation world's own field made the overshoot worse. Its grid signed distance runs from $+1$ to -1 cell across the cell at an obstacle's face, so $|\nabla\phi| = 2$ there. Against it, the barrier is four times and the damper eight times as stiff per metre as against an exact wall. Inside a wall two cells thick the field is flat at -1 cell, so a vehicle that got half a cell in would feel no barrier.

Between two vehicles, each one's points meet the other's outline, a rounded rectangle's signed distance, both ways. Each force acts on both bodies, equal and opposite at the point, so the pair keeps its momentum. The barrier's energy depends only on the pair's relative pose, and it is shared half to each vehicle's books, as the damper's power is. The live books (21) carry three contact lines: the barrier's stored change, the contact's dissipation, and the exchange, what the vehicles passed each other through their contacts. Summed over a pair, the exchange is the integrator's error alone.

Against the rigid-body world. Coasting from 5 m/s into a wall head on, the full-size preset rebounds from the compliant barrier with 0.19 of its approach speed; the rigid-body world's box keeps none of it. At 30° to the wall, the rigid-body world's friction yaws the vehicle 2.8° into the wall, where the compliant plant turns 0.3° the other way, and turns 0.5° toward the wall with its friction raised from 0.2 to 0.6. Into a parked vehicle of the same preset, the compliant plant leaves the two at 1.90 and 2.88 m/s, and the rigid-body world at 2.27 and 2.28 m/s. The compliant contact is the more elastic.

Learning from contact. `cut_flights` with `keep_contact` (`cvc plant-log -keep-contact`) keeps the windows with a contact or an impact, with a last column that counts the contacts each interval began. A learned plant can read it as a port or a modulation and so fit the mean response of the intervals in contact. It cannot learn where or when a contact happens, nor its direction: the rows carry no obstacle. Tables 11 and 12 give the gates.

8 The learned plant: `cvc::ph`

`cvc::ph::se2_vehicle` (`inc/cvc/ph/se2_vehicle.h`) is a native C++ port of an $SE(2)$ vehicle model built on the stochastic port-Hamiltonian Gaussian processes of McLennan, Rapolu and Bajaj [2]. Their JAX code, `CVC-Lab/LieSPHGP`, models an $SO(3)$ pendulum, $SE(3)$ quadrotors and an $SE(3)$ underwater vehicle. It has no $SE(2)$ vehicle.

The port follows a separate $SE(2)$ vehicle package built on that code at commit `65db994`. That package is not part of the public repository. libcvc's golden for the port (`ph_se2_golden.json`) was generated once from it, and its provenance records "`CVC-Lab/LieSPHGP @ 65db994 + the SE(2) package`". The package follows LieSPHGP's underwater-vehicle model reduced to $SE(2)$, with two differences: its input map reads the twist, where LieSPHGP's reads the pose, and its inputs are a vehicle's drive and steering.

The model contains no physical constant: everything is learned from logged runs of a vehicle. In libcvc those runs come from `cvc::physics::plant_log`. It records the vehicles of a navigation world driving the rigid-body engine's plant (`cvc::physics::nav_plant`, on Jolt Physics).

8.1 Model

The state is (x, y, ψ, v) with $v = (v_x, v_y, r)$. The inputs u are the logged commands divided by their training RMS. With $p = Mv$ and $H = \frac{1}{2}p^\top M^{-1}p + V(x, y)$,

$$\begin{aligned} d(x, y) = R(\psi)(v_x, v_y)\, dt, \qquad d\psi = r\, dt, \qquad dv = M^{-1}dp + \Sigma\, dW, \\ dp = \Big(\operatorname{ad}_v^* p - \begin{pmatrix} R(\psi)^\top \nabla V \\ 0 \end{pmatrix} - D(v)\, v + G(v)\, u \Big) dt, \end{aligned} \tag{30}$$

with $\Sigma = \operatorname{diag}(\sigma_1, \sigma_2, \sigma_3)$. Each piece is a learned level plus a Gaussian-process residual:

- $M^{-1} = \operatorname{spd}(\ell_M)$, constant: a rigid body's inertia.
- $D(v) = \operatorname{spd}\left(\ell_D + \mathrm{GP}_D(v/s_v)\right)$, positive definite at every twist: it can only dissipate.
- $G(v) = \ell_G + \mathrm{GP}_G(v/s_v)$, a $3 \times n_u$ input map.
- $V(x, y) = \ell_V \cdot (x, y) + \mathrm{GP}_V\big((x, y)/\lambda_V\big)$, the potential.

Here s_v is the RMS of each twist channel over the training data. Here $\operatorname{spd}(\ell) = LL^\top$, where L is lower triangular with diagonal $e^{\ell_1}, e^{\ell_2}, e^{\ell_3}$ and off-diagonal ℓ_4, ℓ_5, ℓ_6 (`spd_from_levels`, `mat3.h`), so the result is symmetric positive definite for every value of the six levels. Only $M^{-1}D$, $M^{-1}G$ and $M^{-1}\nabla V$ are identifiable from trajectories, plus M's shape through $\operatorname{ad}_v^* p$.

8.2 Gaussian processes on random Fourier features

Each GP is $\mathrm{GP}(z) = \phi(z)^\top W$ on n random Fourier features [5] of a Matérn-ν kernel with length scale λ (`inc/cvc/ph/rff.h`):

$$\phi_k(z) = \sqrt{2/n}\, \cos(\omega_k \cdot z + b_k), \qquad \omega_k = \frac{w_k}{\lambda\sqrt{s_k/(2\nu)}}, \quad w_k \sim \mathcal{N}(0, I),\ s_k \sim \chi^2_{2\nu},\ b_k \sim \mathcal{U}(0, 2\pi). \tag{31}$$

These are the Student-t spectral samples of the Matérn kernel. Every derivative is bounded and in closed form, which the reverse pass uses. The weights have a mean-field Gaussian posterior, $q(W) = \mathcal{N}(\text{mean}, \operatorname{diag} e^{2\log \text{std}})$. The defaults are $n = 64$, $\nu = 5/2$ and $\lambda = 1$. The library's own generator (xoshiro256** with written-out normal and gamma draws) draws the same features from a seed on every compiler.

8.3 Energy balance, output and noise power

Along the drift of (30), $\operatorname{ad}_v^* p$ does no work, and the potential's force returns exactly what moving through V costs. So

$$\dot{H} = -v^\top D(v)\, v + y^\top u, \qquad y = G(v)^\top v. \tag{32}$$

`output()` returns y, so $y^\top u$ is the supplied power. The noise is added to the twist, so it supplies energy too, on average. Since $dH = v^\top M\, dv + \frac{1}{2} dv^\top M\, dv$ (plus the position terms) and Σ is constant, so the Itô and Stratonovich forms agree,

$$\mathbb{E}[dH] = \Big(- v^\top D v + y^\top u + \tfrac{1}{2} \operatorname{tr}\left(\Sigma^\top M \Sigma\right) \Big) dt. \tag{33}$$

`noise_power()` returns $\frac{1}{2}\operatorname{tr}(\Sigma^\top M \Sigma)$. Pathwise, the drift alone is passive. The diffusion's power is booked as its own line, not attributed to D.

8.4 The step

One step of size h is "Lie-IMEX": the damping is implicit and everything else is explicit. With $e = M^{-1}(\mathrm{ad}_v^* p - [R^\top \nabla V; 0] + Gu)$ and $K = M^{-1}D(v)$ evaluated at each stage, and the twist's noise increment $\eta = \sqrt{h}\,\Sigma z$,

$$\begin{aligned} &(I + hK_1)\, v_p = v + h\, e_1 + \eta, && (x,y)_p = (x,y) + hR(\psi)(v_x, v_y), && \psi_p = \psi + hr, \\ &(I + \tfrac{h}{2}K_2)\, v' = v + \tfrac{h}{2}(e_1 + e_2 - K_1 v) + \eta, && (x,y)' = (x,y) + \tfrac{h}{2}\big(R(\psi)v_{xy} + R(\psi_p)v_{p,xy}\big), && \end{aligned} \tag{34}$$

and $\psi' = \psi + \frac{h}{2}(r + r_p)$. It is a Heun predictor–corrector with implicit damping, and so second order. Since $SO(2)$ is abelian, the heading stays a continuous angle without a group-valued update. It is not structure-preserving and keeps no discrete energy identity. The step is templated on the scalar, so the same code runs in float and in double.

8.5 Training

`cvc::ph::se2_rollout_objective` (`inc/cvc/ph/se2_train.h`) is the rollout loss of the $SE(2)$ package's trainer:

$$\mathcal{L} = \mathrm{rollout}(\text{windows} \mid W) + \beta\, \frac{\mathrm{KL}(q \,\|\, p)}{6N_{\mathrm{obs}}}, \qquad W = \mathrm{mean} + e^{\log \mathrm{std}} z,\ z \sim \mathcal{N}(0, I). \tag{35}$$

From each window's first sample, the model is rolled out under the recorded inputs and compared with every later sample (position, wrapped heading and twist): the mean over the window of the squared error summed over the six channels, averaged over the windows that stay finite. The objective takes one reparameterized weight draw per step. β rises linearly to its maximum over an annealing period, and N_{obs} counts the training windows times their intervals. The gradient is a hand-written reverse pass through every step of the window: the 3×3 implicit solves, the SPD parameterization, the random-feature GPs, and ∇V's own derivative in position. The optimizer is LieSPHGP's:

- the whole gradient is clipped to global norm 1;
- Adam, with a cosine-decayed learning rate, updates every leaf except the levels: the GP weights, and the noise scales, although the rollout objective gives the noise scales no gradient (below);
- Prodigy, one per subnetwork, updates the levels.

The command-line tool `cvc ph-train` fits a model to plant-log runs and writes it as JSON (format `cvc-ph-se2/1`).

The rollouts in this objective run the drift alone, with no noise increment. So the objective fits M^{-1}, D, G and V, but Σ gets no gradient from it, and there is no observation model. LieSPHGP's own trainers fit those with an error-state extended Kalman filter: they minimize the filter's negative log marginal likelihood plus the KL term, a negative ELBO, which learns Σ and the observation noise. That stage is not yet ported. The model format already carries the observation-noise parameters it will need.

8.6 Steering: a port in the reference form, a modulation by option

The inconsistency. In the reference form every input is a port. In libcvc's plant logs the inputs are the vehicle's drive and its steering (`plant_log.h`), so the supply $y^\top u$ contains a steering term, $y_\delta u_\delta$. Section 4.3 shows that the vehicle's steering does no work on its body, and nothing in the learned structure forces that term to vanish. With its GP weights zero, the reference form reduces exactly to the linear-tire bicycle with steering as an input (`PhSe2.ReducesToTheLinearTireBicycle`), which is the truncation of Section 4.3.

The diagnostic. Split $y^\top u$ by input channel, using `output()`. Two measurements (transfix/libcvc #851):

- On the golden fit, steering is 13.4% of $|y^\top u|$ at its 16 states. With the drive input at zero, steering alone supplies energy at 10 of them (`PhSe2.TheReferenceFormBooksSteeringAsPower`).
- A fresh reference-form fit to the analytic plant's own flights still books 5.9% of $|y^\top u|$ through steering, and supplies energy by steering alone at 33 of 250 held-out steps, although the plant it learned from books none (`PhSe2Train.BothFormsFitTheDynamicBicycle`). That fit is good: its held-out rollout loss falls from 8.52 to 0.0006. (These are the flights of the plant since #852. On #851's branch, before #852's wheel-frame slip, they were 5.6% and 74 of 250.)

Steering as a modulation. libcvc #851 adds an opt-in form that mirrors Proposition 1: `se2_setup::modulation_dim` (0 by default). With `modulation_dim` $= 1$ the last input, the road-wheel angle δ, stops being a port. It is read by the GPs of D and G beside the twist:

$$D(v,\delta) = \operatorname{spd}\left(\ell_D + \mathrm{GP}_D(v/s_v,\delta)\right), \qquad G(v,\delta) = \ell_G + \mathrm{GP}_G(v/s_v,\delta), \qquad y = G(v,\delta)^\top v, \quad (36)$$

with u the drive alone in (30). The balance (32) holds unchanged, so with the drive at zero nothing is supplied, whatever δ is. Steering still steers, through D and G, as the steered tire's secant does in (9). The modulation is data, like the inputs, so the reverse pass drops its gradient.

- A modulated model is written as `cvc-ph-se2/2`; models without modulations stay `/1`, byte for byte.
- `cvc ph-train -steer-modulates` fits it from plant logs recorded with the actual road-wheel angle, which is `cvc-plant-log`'s default.
- The reference form stays the default and keeps its JAX parity. The modulated form has no JAX golden. It is held to central differences and to the analytic identities (Table 14).
- On the analytic plant's flights, it learns as well (held-out loss $7.63 \to 0.0167$; 0.0092 before #852) with steering's share at zero. The reference form fits those flights more closely, so the modulation is a matter of correct books, not of accuracy.
- At Adam's default learning rate of 3×10^{-3}, the reference form's gradient norm blew up (10^6–10^9) in all three runs tried on that data, while the modulated form converged in all four configurations tried. That is one dataset (the flights before #852) and one seed, so it is reported here, not claimed.

8.7 The potential's form, and the wheels as inputs

The potential $V(x, y)$ of (30) depends on world position, so a fit can tie energy to where the vehicle drove rather than to how (Section 12.1). libcvc #982 makes its form an option, `se2_setup::potential`:

- `world`: the $V(x, y)$ above, and the default;
- `none`: H is the kinetic energy, and the body term is zero;
- `terrain`: the body term is two learned levels times the logged sines of the ground's slope along and across the heading, $(k_x\theta_{\parallel},\ k_y\theta_{\perp})$. That is the gradient of mgz along the body's axes for a vehicle on that slope ($k \to mg$, up to M's scale), read where the vehicle is rather than learned where it was, and its power is the climb's. H cannot evaluate it from (x, y), so `hamiltonian()` leaves it out.

A model whose potential is not `world` is written as `cvc-ph-se2/3`. The plant logs' wide layout (`cvc plant-log -wide`) records the rigid-body simulator's wheel speeds, the pedals, the drive's shaft and brake power, and the ground's slope under the wheels from their contacts. `cvc ph-train -inputs` picks a model's inputs by column name, so the wheels' rim speeds can be ports.

#982 reports a comparison of these options: 192 flights of 5 s in each of two environments, a flat swerve at a mean 12.9 m/s and a city scene with terrain and buildings at a mean 6.0 m/s, scored by the mean model's rollout loss on clean 2 s windows, at seeds 0 and 1. These are training runs, not tests, and they were not repeated for this paper. Trained on the city scene, the ground-slope term had the best held-out loss, 0.472 and 0.441, against $V(x, y)$'s 0.499 and 0.467 and no potential's 0.503 and 0.465. The body's attitude, used as the slope instead, fits the suspension's pitch under acceleration on flat ground and fails on slopes, which is why the logs take the ground's slope from the wheels' contacts. The rim speeds as ports overfit the city scene (held-out 0.58–0.67). Between the two environments, the difference in speed dominates every model.

9 The navigator's energy identity and context work

The plant's books say where the *vehicle's* energy goes. The navigator has its own energy, $H_{\text{nav}} = \frac{m}{2}\|v\|^2 + U_{\text{nav}}$, on its point-mass surrogate (`integrate_surrogate_material`). libcvc #832 adds per-step books for that surrogate, and books across ticks for the deployed drive's potential.

9.1 The per-step identity

One semi-implicit step of a point mass m in a potential U held fixed over the step (coefficients frozen), with damping γ, a port force F and a speed clamp, is

$$v' = v + \tfrac{h}{m}(-g - \gamma v + F), \qquad \tilde{v} = \text{clamp}_{v_{\max}}(v'), \qquad x' = x + h\tilde{v}, \qquad g = \nabla U(x). \tag{37}$$

With $H = \frac{m}{2}\|v\|^2 + U$, define the damping work $W_\gamma = \frac{h}{2}\gamma\, v\cdot(v+v')$, the port work $W_{\text{port}} = \frac{h}{2}F\cdot(v+v')$, the clamp loss $W_{\text{clamp}} = \frac{m}{2}(\|v'\|^2 - \|\tilde{v}\|^2) \geq 0$ and the clamp cross term $W_\times = h\, g\cdot(\tilde{v} - v')$. Then the residual is

$$r_E = \Delta H + W_\gamma + W_{\text{clamp}} - W_{\text{port}} - W_\times = \frac{h^2}{2}\Big[\tilde{v}^\top \bar{\mathcal{H}}\, \tilde{v} - \frac{1}{m}\, g\cdot(g + \gamma v - F)\Big], \tag{38}$$

exactly, with $\bar{\mathcal{H}} = 2\int_0^1 (1-s)\nabla^2 U(x + s\,\Delta x)\, ds$ the averaged Hessian. (Expand $\frac{m}{2}(\|v'\|^2 - \|v\|^2) = \frac{h}{2}(-g - \gamma v + F)\cdot(v + v')$, and Taylor-expand $U(x') - U(x) = h\, g\cdot\tilde{v} + \frac{h^2}{2}\tilde{v}^\top\bar{\mathcal{H}}\tilde{v}$.) `book_step` books

any such step from its quantities. `rollout_energy` carries the books for every step of the material surrogate. They are evaluated in double, so they resolve terms of order h^3 below float round-off on U.

9.2 Two checks

- *The cheap bound*, every step:

$$|r_E| \le \kappa_H \frac{h^2}{2}\Big[\frac{|g|\,(|g| + \gamma|v| + |F|)}{m} + \hat{L}\,|\tilde{v}|^2\Big],$$

  with the secant Lipschitz estimate $\hat{L} = |\nabla U(x') - \nabla U(x)|/|x' - x|$ and safety factor $\kappa_H = 2$ (`energy_residual_bound`). It is a tolerance model, not a theorem, since $\hat{L}$ estimates the curvature and does not bound it. It catches work of order h left out of the books: an unlogged force, a stale field, an unbooked clamp.

- *The sharp check*, when asked (two extra force evaluations a step): $\hat{r}_E = \frac{h^2}{2}\big[\tilde{v}^\top \nabla^2 U(x)\tilde{v} - g \cdot (g + \gamma v - F)/m\big]$, with $r_E - \hat{r}_E = O(h^3)$. It catches a second-order error the identity does not predict. Explicit Euler ($x' = x + hv$) is the example: its r_E sits $h^2 g \cdot (g + \gamma v - F)/m$ above $\hat{r}_E$. That is second order, the same order as the bound, so explicit Euler passes the bound and fails the sharp check.

The explicit-Euler case mattered in practice. Through libcvc 3.7.11, `cvc::nav`'s native multi-start rollout ran explicit Euler by default, where its Python namesake runs semi-implicit, as GRL-SNAM's guide noted. On circular orbits of a goal spring, `NavPotential.ExplicitEulerRaisesTheAlarm` shows explicit Euler gaining energy at every step, by $\omega^2 h^2$, with the sharp check at half the bound on every step. The semi-implicit step meets the sharp check to 3.5×10^{-5} of the bound.

The integrator. libcvc's geometry rollout (`integrate_surrogate_v2`, which the material trainer's multi-start loss uses) now defaults to semi-implicit Euler (37), as every other point-mass rollout already did: the velocity updates first, and the position steps with the new velocity (transfix/libcvc #981). Explicit Euler remains an option, and the sharp check still alarms on it. On `nav_geom_rollout_grad_test`'s penetrating batch the multi-start penalty is 8% lower under semi-implicit Euler on the legacy start (2.050 against 2.230) and 15% lower on sampled starts (2.453 against 2.898).

#981 also reports an A/B, a training run outside CI that was not repeated for this paper. It trained the coefficient policy on the point mass under each integrator, ten training seeds per arm on the city GRL-SNAM trains on. It then drove every policy on the rigid-body simulator's 1/10-scale vehicles, over 20 seeds of 48 agents, with paired bootstrap 95% intervals.

- Where training moves the policy (learning rates 10^{-4} and 2×10^{-4}), explicit Euler costs reach: -7.3 $[-10.1, -4.7]$ and -4.7 $[-8.4, -1.6]$ points.

- At the gate's learning rate, 5×10^{-5}, the arms barely differ: explicit Euler gains $+1.0$ $[+0.1, +1.9]$ points of reach, but drives 5% further and hits buildings 4% more.

- On the trained policies' point-mass steps, booked outside every barrier, the sharp check never alarms under semi-implicit Euler (its largest ratio is 1.2×10^{-3}). Under explicit Euler it alarms on 14–19% of steps, and 78–94% of those gained energy.

The alarm fires on a minority of steps because the trained point mass is heavily damped and near its terminal velocity, where $g + \gamma v - F \approx 0$ and explicit Euler's excess $h^2 g \cdot (g + \gamma v - F)/m$ vanishes. A Strang splitting, velocity Verlet with the damping and the speed clamp split out, never differed from semi-implicit Euler outside the interval and was not kept. GRL-SNAM's native material trainer leaves the integrator to libcvc on its legacy start, so with a `pycvc` that carries #981 that start runs semi-implicit, while its torch twin keeps explicit Euler as its default.

9.3 Context work and projection work

Across a tick of the deployed drive, U_{nav} is not fixed. The occupancy map is re-sensed, the field rebuilt, the coefficients re-predicted, and the carrot moved. Those changes do work on the navigator's energy that no step's books see. `sim_world` books them in its drive telemetry while it captures:

- `potential_end`: U at the pose the tick ends at, against the tick's field;
- `projection_work`: U's change across anything that moves the agent after the drive, the kinematic override and the `min_gap` de-overlap: `potential_end` minus U where the drive (or the plant) left the agent;
- `context_work`: U at the tick's start pose with this tick's context, minus U at the same pose with last tick's context (coefficients, carrot, field, material planes), evaluated as `step()` begins.

When nothing moves the agent between ticks, `context_work` equals this tick's starting U minus last tick's `potential_end`, bit for bit. The potential's change over a run then telescopes:

$$U_K^{\text{end}} - U_1^{\text{start}} = \sum_{k=1}^{K} \left(U_k^{\text{drive}} - U_k^{\text{start}}\right) + \sum_{k=1}^{K} \text{projection}_k + \sum_{k=2}^{K} \text{context}_k, \tag{39}$$

where U_k^{drive} is U where the drive left the agent on tick k. The first sum is what the drive's motion did under a fixed context. The other two are what the runtime did to the landscape. The drive itself leaves the three fields zero. The bicycle and dynamic rollouts are not point masses, so they reject requests for per-step point-mass books (they throw). For a vehicle, the plant's books of Section 5 are the counterpart.

9.4 Two sets of books, not one

When a navigation world drives the dynamic plant, there are two sets of books. The plant books the vehicle's kinetic energy (21). The telemetry books the navigator's potential (39). They are not one conserved quantity. U_{nav} is a learned landscape whose coefficients change every tick, not a physical energy of the vehicle. The two meet only through the controller, which is not a power-conserving interconnection. Keeping both, with their own residuals, is what makes each one checkable.

10 GRL-SNAM's side

10.1 The twins and their parity tiers

GRL-SNAM keeps Python twins of libcvc's vehicle and potential code. libcvc writes a golden file from its own test and GRL-SNAM copies it into `tests/fixtures`. The twin is then held to the golden at a stated tier:

- *double*: both sides in float64 from the same parameters, to 10^{-9} relative;

- *float*: libcvc in float32 against the twin in float32, to about 10^{-5};
- *mixed*: libcvc's float rollouts against the twin in float64, to 10^{-4} on values and 10^{-3} on gradients.

The torch twins differentiate with autograd, where libcvc uses forward-mode duals or hand-written reverses. So the gradient parity compares two independent derivations of one model. Tables 1 and 2 list the twins on `main`, the second those of Section 7.

The JAX reference for the learned plant runs the other way. The $SE(2)$ vehicle package built on `CVC-Lab/LieSPHGP` (Section 8) generated libcvc's golden (`ph_se2_golden.json`) once, and the C++ port is held to it.

The torch tests need only torch and the copied goldens, not `pycvc`. (Two tests in `test_material_ext_parity.py` also hold `pycvc`'s bindings to the golden, and skip when the installed `pycvc` lacks the external-force input.)

10.2 What is not twinned yet

The pH form's parts, rates and step books are twinned in CVC-Lab/GRL-SNAM #138, held to libcvc's `DynBikePH.Golden`; the tangent `dyn_ph_tangent_D` is not. GRL-SNAM #146 twins the extensions of Section 7: both wheel-spin plants with their forms and books, the single track's compliant contact, and its full form with the side slope. The four-wheel vehicle's other forms (Section 6) are not twinned: `grl_snam.dyn4` twins `dyn4_plant` and `dyn4_spin_*`, not `dyn4_ph_*`, `dyn4_plant_books` or the four-wheel full form, and it raises if a plant asks for the grade force. The navigator's point-mass books (`book_step`, the cheap bound, the sharp check and `rollout_energy`'s book fields) are twinned in #139, held to libcvc's `NavPotential.BooksGolden`. The tick telemetry (`potential_end`, `projection_work`, `context_work`) is a running world's, not a function's, so it has no twin. libcvc #857 binds it in `pycvc` instead: `nav_sim_world_set_capture_drive_telemetry` and `nav_sim_world_drive_telemetry` read a `sim_world`'s last tick, the three books included. #857 merged as `713a14af`, and `pycvc` carries it from libcvc 3.7.2 on. `pycvc` reports a world's plant kind (`nav_sim_world_plant_kind` returns `"dynamic"`), but not the plant's books. #831 left `dyn_plant`'s outputs untouched. libcvc #852's wheel-frame front slip changes them, and #138 moves the twin's plant with it, along with its three fixtures (`dyn_bike_golden.json`, `dyn_bike_ph_golden.json`, and the dynamic case of `material_ext_golden.json`).

10.3 Using the plant and its books from GRL-SNAM

What follows is what GRL-SNAM can do with what is merged today, and the small pieces it would add. Pieces not yet built are marked as proposals.

Train through the dynamic plant. The dynamic rollouts are twinned and differentiable: `integrate_dynamic_material` for the material bicycle, and `dyn4_rollout_ext` for the four-wheel vehicle with an external force. A trainer that today rolls out the kinematic bicycle can roll out `dyn_bike.dyn_plant` in its place, under the same `bike_control` commands, and backpropagate through tires, load transfer and actuator lag. In C++ the same choice is `coef_train`'s `rollout_kind::dynamic`. In a live world it is `plant_host.use(sim, "dynamic")`.

Book the energy. The torch twin of the books (#138) is small, because `dyn_bike.py`'s step already computes every part: the axle forces $F_{x,w}$, the lateral forces $F_{y,w}$, the secants S_w and the forward speeds u_w. The rates at a state (`dyn_ph_rates_at`) are

Twin	libcvc reference; golden; test	Tier and gate	Added
`grl_snam/dyn_bike.py dyn_plant`	`diff::dyn_plant` and its dual-number VJP; `dyn_bike_golden.json` (`DynBike.Golden`, 7 cases: cruise, hard corner, braking in a turn, past the peak, reversing, creeping, pulling away); `tests/test_dyn_bike_parity.py`	double: next state and VJP to 10^{-9} relative	GRL-SNAM #121; golden #134; wheel-frame slip #138
`grl_snam/dyn_bike.py dyn_ph_parts_at`, `dyn_ph_rates_at`, `dyn_plant_books`	`dyn_ph_parts_at`, `dyn_ph_rates_at`, `dyn_plant_books`; `dyn_bike_ph_golden.json` (`DynBikePH.Golden`, the 7 cases and a stop at the speed clamp); `tests/test_dyn_bike_ph_parity.py`	double: parts, rates, books and the books' gradient to 10^{-9} relative	#138; golden libcvc #848
`grl_snam/dyn4.py dyn4_plant`	`diff::dyn4_plant` (four wheels, suspension); `dyn4_golden.json` (11 cases); `tests/test_dyn4_parity.py`	double, 10^{-9}	#123
`grl_snam/ext_rollout.py dyn4_rollout_ext`	`diff::dyn4_rollout_ext`, with an external force; `dyn4_ext_rollout_golden.json`; `tests/test_dyn4_ext_rollout_parity.py`	mixed: states 10^{-4}, gradients 10^{-3}	#136
`grl_snam/material_bike.py integrate_dynamic_material`	`integrate_dynamic_material`; the dynamic case of `material_ext_golden.json`; `tests/test_material_ext_parity.py`	mixed: forward 10^{-4}, gradients 10^{-3}	#136; regenerated #138
`grl_snam/nav_potential.py`	`detail/nav_potential.h`; `nav_potential_golden.json` (`NavPotential.Golden`: barrier sweeps, U along a material rollout, `drive_potential`); `tests/test_nav_potential_parity.py`; also $F = -\nabla U$ by autograd	float: 2×10^{-5} (5×10^{-5} along the rollout)	#133
`grl_snam/nav_potential.py book_step`, `energy_residual_bound`, `material_energy`; `integrate_surrogate_material`'s books	`book_step`, `energy_residual_bound`, `rollout_energy`'s book fields; `nav_energy_books_golden.json` (`NavPotential.BooksGolden`: the bound on a sweep, a `diff::surr_step` rollout whose speed clamp cuts every step, the material surrogate with a port force and the sharp check, an unlogged port); `tests/test_nav_energy_books_parity.py`	double: `book_step` on libcvc's arguments to 10^{-12}; float: books to 10^{-5} of the step's bound (2×10^{-5} clamped)	#139; golden libcvc #849
JAX reference: the $SE(2)$ package built on `CVC-Lab/LieSPHGP` at `65db994` (not public)	that package against `cvc::ph::se2_vehicle` and `se2_train`; `ph_se2_golden.json`; libcvc `ph_se2_test`, `ph_se2_train_test`	double: one step 10^{-13}, rollout 10^{-12}, gradient 10^{-10}	libcvc #758

Table 1: GRL-SNAM's twins of libcvc's vehicle and potential code, and the JAX reference for the learned plant. GRL-SNAM numbers are CVC-Lab/GRL-SNAM pull requests.

Twin	libcvc reference; golden; test	Tier and gate	Added
`grl_snam/dyn_bike.py` `dyn_spin_plant`, `dyn_spin_ph_parts_at`, `dyn_spin_ph_rates_at`, `dyn_spin_plant_books`	the wheel-spin plant, its pH form and books; `dyn_spin_ph_golden.json` (`DynSpinPH.Golden`, 9 cases, among them a wheelspin launch, trail braking that locks the front, and a locked skid); `tests/test_dyn_spin_ph_parity.py`	double: next state, parts, rates, books and the books' gradient to 10^{-9} relative	#146; golden libcvc #982
`grl_snam/dyn4.py` `dyn4_spin_plant`, `dyn4_spin_ph_parts_at`, `dyn4_spin_ph_rates_at`, `dyn4_spin_plant_books`	the four-wheel wheel-spin plant, its form and books on any ground; `dyn4_spin_ph_golden.json` (`Dyn4SpinPH.Golden`, 7 cases); `tests/test_dyn4_spin_ph_parity.py`	double, 10^{-9}	#146; golden #982
`grl_snam/dyn_contact.py`; `dyn_plant(..., wrench=...)`	`nav/detail/dyn_contact.h`, and `dyn_plant`'s external port with its books' port line; `dyn_contact_golden.json` (`DynContact.Golden`: the footprint, field cases against a wall and a pillar, pair cases, a plant step); `tests/test_dyn_contact_parity.py`	double, 10^{-9}	#146; golden #983
`grl_snam/dyn_bike.py` `dyn_ph_rhs_full`, `dyn_plant_ground`, `dyn_plant_books_ground`	the full form, the ground overloads, and the side-slope plant and books; `dyn_ph_full_golden.json` (`DynPHFull.Golden`, 6 states on a hillside); `tests/test_dyn_ph_full_parity.py`	double: values, and the books' gradient through the ground, to 10^{-9}	#146; golden #987

Table 2: GRL-SNAM's twins of the extensions of Section 7. GRL-SNAM numbers are CVC-Lab/GRL-SNAM pull requests.

```
nf = (-sin d, cos d, lf cos d);  nr = (0, 1, -lr)
tf = (cos d, sin d, lf sin d);   tr = (1, 0, 0)
R  = cx e_x e_x^T + Sf nf nf^T + Sr nr nr^T
supplied   = v . (tf Fxf + tr Fxr)          # y^T u
dissipated = v^T R v                        # >= 0
floor      = v . (nf (Fyf + Sf nf.v) + nr (Fyr + Sr nr.v))   # round-off
grade      = -m g theta vx
```

The books of a step (`dyn_plant_books`) are the trapezoid of these between the start (with the step's actuator outputs) and the pre-limiter twist, plus H's change across the limiters (Section 5). `tests/test_dyn_bike_ph_parity.py` holds them to libcvc's `DynBikePH.Golden` in double: the worst relative difference is 7.1×10^{-14} on the books and 4.4×10^{-14} on their gradient, which autograd takes and libcvc takes by dual numbers.

Book the navigator's energy. The point-mass surrogate's books come with its rollout. Given `energy={"books": True, "sharp": True}`, `integrate_surrogate_material` fills `kinetic`, `w_damp`, `w_port`, `w_clamp`, `w_cross`, `residual`, `bound` and `residual_sharp` for every step, in float64 and detached, as libcvc's `rollout_energy` does. The rollout's outputs are unchanged bit for bit. `nav_potential.book_step` books any other such step; the parity test books `sdf_rollout`'s clamped surrogate with it. `tests/test_nav_energy_books_parity.py` holds the books to `NavPotential.BooksGolden`. `book_step` on libcvc's arguments agrees bit for bit, and the books agree to 1.1×10^{-6} of the step's bound on the clamped rollout and 8.3×10^{-8} on the material surrogate. The books see only the integrator when the patch's gradient channels are its values' gradients. Otherwise each step also books the gap between the force and $-\nabla U$, work of order h, which the cheap bound reads as a stale field: on

`NavPotential.Golden`'s curved patch it fills 99.65% of the bound at $h = 0.002$. Across ticks, a live world's books come from `pycvc` (libcvc #857). With capture on, `nav_sim_world_drive_telemetry` gives each agent's `projection_work` and `context_work`, the terms of the last two sums of (39).

Energy-aware objectives (proposal). With the books available inside the rollout, a trainer can price energy directly:

- $\sum \max(y^\top u, 0)\, h$, the drive energy, as a cost;
- the dissipated energy $\sum v^\top \mathsf{R} v\, h$ as a proxy for tire scrub and sliding (it grows sharply past the peak);
- the limiter energy as a penalty for leaning on the speed clamp.

Each is differentiable almost everywhere, as the plant is. Because steering does no work, the drive energy counts every joule the policy puts into the vehicle.

Health checks. The books double as run-time checks of a rollout:

- the residual's share of the flow (in the tests' runs, 1.1×10^{-4} to 1.2×10^{-3} at $h = 0.01$ s) flags a step too long for the motion;
- a floor line above round-off flags a plant whose slips no longer root at $n_w^\top v = 0$;
- a limiter line that dominates flags a policy that lives at the speed clamp.

On the navigator side, the cheap bound flags an unlogged force or an unbooked clamp, and the sharp check flags an explicit-Euler integrator.

New force terms are ports. GRL-SNAM's research extensions add force terms to the navigator: material and terrain costs, and external forces. Section 4.8 gives the rule. A conservative term goes in U_{nav} with its force $-\nabla U$, which the twin's autograd gradcheck verifies. A non-conservative term is a port, booked as W_{port} on the surrogate. On the vehicle, it is either already in $y^\top u$ (if it acts through the controller) or a new column G_e (if it acts on the body).

11 Validation

Tables 3–16 map each claim to the test that checks it. They give the tolerance the test enforces and the measured value. The commit is the squash commit of the pull request that added or last tightened the test.

Where the values were measured (Windows, MSVC 19.44, double unless stated; Appendix A gives the exact setup):

- libcvc `c1f104c0`: all 62 tests of the five executables passed. The tables list the ones this paper relies on.
- The rows for #851: its branch commit, where the same executables' 69 tests passed (plus the CLI's four `PhTrain` tests). Its `ph` sources and tests are those of its squash commit `b8ac9975`, but the branch predates #852, so the analytic plant it fits in `PhSe2Train.BothFormsFitTheDynamicBicycle` had the old front slip. That row is measured on `master` at `6cc8cfdc` instead, whose `ph` sources and tests and whose single-track plant are `b8ac9975`'s.

- GRL-SNAM `0e21085` (torch 2.8 CPU): 53 tests passed and 2 were skipped for the `pycvc` reason above; none failed.

- The rows for libcvc #852 and GRL-SNAM #138: libcvc's branch commit `1a0acc77`, where `nav_dyn_bike_test`'s 23 tests and `physics_dyn_bike_plant_test`'s 3 passed, and so did the ten other suites that use the plant; and GRL-SNAM's `2f76ab0`, #138's squash commit, where the dynamic twins' parity tests passed (57 passed, 6 skipped for `pycvc`). `1a0acc77`'s `nav` and `physics` sources and tests are those of #852's squash commit `140bc2d0`.

- The rows for #850 (Table 8): libcvc `master` at `6cc8cfdc`, where `nav_dyn_4w_test`'s 23 tests passed. Its pH form and books are those of #850's squash commit `45dab152`; a later change (#940) adds a drive response to the plant that passes the command through by default, and the plant's goldens pass unchanged.

- The rows for #869 and #873 (Table 7): libcvc `master` at `6cc8cfdc`, where `physics_dyn_bike_plant_test`'s 8 tests passed. Its `physics` sources and tests are those of `master` at `09dcfa76`.

- The rows for #982, #983 and #987 (Tables 9 and 10, the values before #1004 in Table 12's caption, and the potential's row of Table 14): libcvc `master` at `3d5c15c6`, whose `nav`, `ph` and `physics` sources and tests are those of #987's squash commit `b89992ef`. There `nav_dyn_spin_test`'s 13 tests passed, `nav_dyn_ph_full_test`'s 8, `physics_dyn_contact_test`'s 7, `physics_dyn_bike_plant_test`'s 10 and `ph_se2_train_test`'s 13, and so did `nav_dyn_bike_test`'s and `nav_dyn_4w_test`'s 23 each, the defaults' goldens among them. Table 7's rows measure there as they did on `6cc8cfdc`. The fast impact before #1004 (Section 7.3) is that commit's plant, driven from Python through libcvc #1002's binding of the new options (`01d77192`, on `3d5c15c6`; Section 12), on `make_sim`'s world in libcvc's `test_pycvc_physics.py` with a wall across the road.

- The rows for #1004 (Tables 11 and 12, and Section 7.3's fast impact) and for #981 (Table 15's last three rows, and the penalties of Section 9): libcvc `master` at `352f1933`, where every file the two PRs touched is as their squash commits (`ce9aebaf`, `45cb239a`) left it. There `physics_dyn_contact_test`'s 9 tests passed, `physics_dyn_bike_plant_test`'s 10, `nav_geom_rollout_grad_test`'s 16, `nav_coef_train_test`'s 30, `nav_material_train_test`'s 18 and `nav_potential_test`'s 17. `DynContact.Golden`'s file is byte-identical to GRL-SNAM's fixture, so the contact twin stands.

- The rows for #978 (Table 4 and Section 5.3): libcvc `master` at `f94a9b27`, where #978's files are as its squash commit `789d2f6e` left them, `nav_dyn_bike_dg_test`'s 7 tests passed, and `nav_dyn_bike_test`'s 23 still pass.

- The rows for #1045 (Table 5 and Section 5.3): libcvc `master` at `fcfc4ee7`, #1045's squash commit, where `nav_dyn_bike_dg_test`'s 13 tests passed, #978's 7 among them. The costs are the range of five runs on a shared build host.

- The GRL-SNAM #146 rows: `main` at `41ddf84`, where the four parity files of #146 passed all 65 of their tests.

Appendix A gives the exact versions, the commands that produced these values, and the lines each test prints.

Test	What it shows	Tolerance	Measured	Commit
`DynBikePH.TheStructure IsPortHamiltonianAt EverySlip`	J skew, R symmetric with non-negative coefficients, $w^\top \mathsf{R} w \geq 0$, $H = \frac{1}{2} p \cdot v$; no supply without axle force (Prop. 1, Sec. 4.3); 500 states at each of $\mu = 0.9, 0.6$, steer to 0.5 rad, forward and reversing	exact for skew, symmetry and zero supply; $v^\top \mathsf{J} v$ to 10^{-9}; at least 100 states past a peak	holds at all 1000 states; 244 of 500 past a peak at each μ	`950154b` (#831)
`DynBikePH.TheVector FieldIsThePlantsAtEvery Slip`	(13) is the plant's vector field: a 10^{-5} s `dyn_plant` step against `dyn_ph_rhs`, flat and on 3%, forward and reversing, 400 states per μ	10^{-3} of mg (mgL on yaw)	worst $1.6 \times 10^{-4} mg$ ($\mu = 0.9$), $1.1 \times 10^{-4} mg$ ($\mu = 0.6$)	`950154b` (#831)
`DynBikePH.StepBooks CloseToSecondOrder`	the books (18) close to second order per step (Prop. 2); the limiters are charged when braking to a stop in a turn	per-halving ratio in $(3, 5.5)$; summed residual below 5×10^{-3} of the flow for $h \leq 0.01$	ratios 3.83, 3.92, 3.95; 1.2×10^{-3} at $h = 0.01$; braking: limiter -358 J, residual 4×10^{-6}	`950154b` (#831), `140bc2d0` (#852)
`DynBikePH.TheFloorTerm IsRoundOff`	f_ε is the secants' rounding (Sec. 4.4)	10^{-12} of the flow, $\varepsilon = 0.5$ to 10^{-9}	at most 2.2×10^{-16}	`140bc2d0` (#852)
`DynBikePH.TheTangentIs TheTiresDerivative`	(15)–(16) is $-\partial(Q_y, Q_r)/\partial(v_y, r)$ at 300 states	10^{-6} (central differences)	1.4×10^{-7}	`140bc2d0` (#852)
`DynBikePH.TheSecant StaysPSDWhereTheTangent DoesNot`	tangent indefinite past the peak, secant never (Sec. 4.6)	tangent PSD below $0.5\mu g$; indefinite somewhere above; rear never past peak; secant never negative	as stated at $\mu = 0.9$ and 0.6; indefinite in 25–50% of samples above $0.5\mu g$	`950154b` (#831)
`DynBike.Golden`	`dyn_plant`'s outputs: unchanged by the books (#831), regenerated for the wheel-frame slip (#852)	golden values	holds	`1e3ed0b` (#714), `140bc2d0` (#852)
`DynBike.KinematicLimit FollowsBikeStep`	the kinematic bicycle is the stiff-tire limit, with the mass at the rear axle	worst gap below 0.2 of the vehicle radius; closed-loop median below 0.1 of it	within 0.011 of the kinematic path (mean path 5.6)	`1e3ed0b` (#714)

Table 3: The analytic plant: its pH form and its books (libcvc's `nav_dyn_bike_test`). The flow is the sum of the magnitudes of every line of the books. #714 is the plant itself, which predates the pH work. Before #852 (on `c1f104c0`) braking measured -354 J and 7×10^{-7}, and the floor test, then `TheFloorTermVanishesWithTheFloor`, measured $5.4 \times 10^{-4} \to 5.4 \times 10^{-6} \to 5.4 \times 10^{-8}$; the other rows' values are unchanged.

Test	What it shows	Tolerance	Measured	Commit
`DynBikeDG.TheRateIsThePortHamiltonianField`	the step's momenta's rate is (13)'s field (`dyn_ph_rhs`) at every slip, $\mu = 0.9$ and 0.6	10^{-12} of mg	6.7×10^{-16}, 1.1×10^{-15}	`789d2f6e` (#978)
`DynBikeDG.TheActuatorsAreDynPlants`	the actuators' outputs are `dyn_plant`'s, in double and in float	bit for bit	bit for bit	`789d2f6e` (#978)
`DynBikeDG.StepBooksCloseExactly`	the books (18) close to the solve's tolerance on Proposition 2's swerve, $h = 0.06$ to 0.0025 s, and braking to a stop through the clamp and the blend; every Newton solve converges	10^{-10} of the step's flow; mean below 10^{-6} of `dyn_plant`'s; limiters charged	at most 1.7×10^{-10} J a step (3.5×10^{-14} to 1.3×10^{-12} of the flow), against 25 to 0.056 J; braking at most 5.9×10^{-11} J; 3 to 4 Newton steps	`789d2f6e` (#978)
`DynBikeDG.SecondOrderWhereDynPlantIsFirst`	the error at 2 s of a turn-in past the peak, per halving of h from 0.04 s	ratio in $(3.4, 4.6)$; `dyn_plant`'s below 2.6	3.92, 3.97, 4.03 (5.3×10^{-3} at 0.04 s); `dyn_plant` 1.98, 2.04, 2.13 (0.45)	`789d2f6e` (#978)
`DynBikeDG.StiffTires`	coasting at full lock from 0.5, 3 and 12 m/s, tires 1×, 4× and 16×, one 0.06 s step or six 0.01 s, with and without the low-speed blend: no step gains energy	gain below 10^{-12} of H; H never rises	every step's gain exactly 0; at six 0.01 s substeps with the vehicle's tires, slip swings 0, 1 and 1, as `dyn_plant`'s	`789d2f6e` (#978)
`DynBikeDG.StiffTiresReachTheKinematicBicycle`	a steady 5 m/s turn with 16× tires reaches the kinematic bicycle's yaw rate	2%	−0.02% (`dyn_plant` −0.03% at 0.06 s)	`789d2f6e` (#978)

Table 4: The opt-in discrete-gradient step (Section 5.3; libcvc's `nav_dyn_bike_dg_test`), measured on `master` at `f94a9b27`. Its `CostPerStep` has no gate; there a step took 2.8 to 3.1 μs against `dyn_plant`'s 0.30 μs.

Test	What it shows	Tolerance	Measured	Commit
`DynSpinDG.TheRateIsThePortHamiltonianField`	the step's rate is `dyn_spin_ph_rhs` at 500 states (rolling, spinning, locked, reversed; grades 0 to 0.08; $\mu = 0.9$, 0.3); a wrench enters the body's rows alone; the Jacobian's one-tire rate is the same	10^{-12} of mg	8.2×10^{-15}, 2.0×10^{-15}; wrench 3.3×10^{-16}; one-tire 1.1×10^{-14}	`fcfc4ee7` (#1045)
`DynSpinDG.TheActuatorsAreDynSpinPlants`	the actuators' outputs are `dyn_spin_plant`'s, lags and slew limit on, double and float	bit for bit	bit for bit	`fcfc4ee7` (#1045)
`DynSpinDG.StepBooksCloseExactly`	the books close to the solve's tolerance, wheels' rows at the end and at the midpoint: the swerve past the peak, a launch and lock ($\mu = 0.3$, slip ratio to 8.8), the swerve against a wrench; $h = 0.06$ to 0.0025 s; every solve converges	10^{-10} of the step's flow; `scheme` at least 0, and 0 at the midpoint	at most 3.3×10^{-10} J a step (2.4×10^{-15} to 1.4×10^{-12} of the flow), against `dyn_spin_plant`'s mean 20 to 240 J at 0.06 s; `scheme` at 0.06 s: 21 J (swerve), 4.2×10^{4} J (launch; slip 4.4×10^{5} J)	`fcfc4ee7` (#1045)
`DynSpinDG.Order`	the error at 2 s of a turn-in from 10 m/s, per halving of h from 0.04 s and from 2 ms	wheels at the end: ratio in $(1.7, 2.6)$, error below 0.2 of `dyn_spin_plant`'s; midpoint: $(3.4, 4.6)$	end: 2.30, 2.17, 2.20 (2.4×10^{-2} at 0.04 s), 10 to 13 times below `dyn_spin_plant`'s (2.01, 2.06, 2.14; 0.24); midpoint: 4.15, 4.09, 4.10; from 2 ms, 2.03 to 2.14 and 4.01 to 4.05	`fcfc4ee7` (#1045)
`DynSpinDG.StiffWheelSlip`	one 0.06 s step, blend off: braking from 20 m/s, driving on $\mu = 0.3$, coasting from 0.5, 3 and 12 m/s with the rims 1.5 times the ground	wheels at the end: swings at most `dyn_spin_plant`'s plus 3; coasting, no gain above 10^{-12} of H	swings 2, 0, 2, 3, 6 at the end; 6, 0, 98, 98, 98 at the midpoint (both wheels, every step); `dyn_spin_plant` 2, 0, 0, 1, 4. Braking ends -0.11 m/s off a 0.1 ms run (-0.38), spinning's rear rim -0.56 m/s (-4.98). No step gains energy	`fcfc4ee7` (#1045)

Table 5: The opt-in discrete-gradient step with the wheels spinning (Section 5.3; libcvc's `nav_dyn_bike_dg_test`), measured on `master` at `fcfc4ee7`. The wheels' rows take the step's end by default. Its `CostPerStep` has no gate; over five runs on a shared build host a step took 11 to 16 μs at 0.06 s and 6.3 to 9.0 μs at 0.01 s, against `dyn_spin_plant`'s 0.42 to 0.71 μs.

Test	What it shows	Tolerance	Measured	Commit
`DynBikePlant.StepsAsThe DynamicPlantDoes`	the live plant's ticks are `dyn_plant`'s substeps	bit for bit	bit for bit	`eb9187c` (#833)
`DynBikePlant.TheBooks Close`	(21) closes on two vehicles over 30 s of weaving and braking	identity 10^{-9} of the flow; residual 5×10^{-3}	residual 1.1×10^{-4} (full-size), 8.9×10^{-4} (1/10 scale) of the flow	`eb9187c` (#833)
`DynBikePlant.PlantHost PutsTheAgentsOnItByName`	`plant_host` puts a world on `"dynamic"` with no physics world; its books close	every agent reaches its goal; books as above	3 agents reach goals 120 m on in 11.5 s	`eb9187c` (#833)

Table 6: The live plant (libcvc's `physics_dyn_bike_plant_test`). Its floor line is 5×10^{-15} J over the full-size run since #852 (1.1 J before); the residuals are unchanged.

Test	What it shows	Tolerance	Measured	Commit
`DynBikePlant.ClimbsThe Ground`	6 s up a 5% ramp, without the driver: the speed lost, and the grade line against $mg\,\Delta z$	3%; 2%; identity 10^{-9} of the flow	2.864 m/s against $g \sin(\text{slope})\, t = 2.938$; -1.986×10^4 J against -1.992×10^4 J	`5ca26cf6` (#869)
`DynBikePlant.TheDriver HoldsAndClimbs`	on a 10% ramp with nothing asked, the driver holds; asked for 1 m/s^2 up it, it delivers	held within 0.05 m; 3%	held -0.033 m (the raw command rolls -15.3 m); 5.87 m/s at 6 s against 6.0 (raw 0.051)	`462d8229` (#873)
`DynBikePlant.Collides WithTheWorld`	driven into a wall 30 m on: the nose stops at it, contact takes energy out, the books close	nose within 1 m short to 0.05 m past; contact < 0; identity 10^{-9} of the flow	nose at 30.000 m; 684 colliding substeps; contact -1.39×10^5 J against drive 1.51×10^5 J; 146.0 m without the world	`5ca26cf6` (#869)
`DynBikePlant.Slides AlongAWall`	steered into a wall at 3 m/s for 10 s: the chassis box's friction against it	over 20 m along it at 0.3; under 10 m at 1	26.7 m at 0.3 (the default); 4.4 m at 1	`462d8229` (#873)

Table 7: The live plant's ground, world and driver (libcvc's `physics_dyn_bike_plant_test`, measured on `master` at `6cc8cfdc`, where its 8 tests pass). There, with the driver on, `DynBikePlant.TheBooksClose`'s run closes with residuals of 1.68×10^{-4} (full-size) and 6.45×10^{-4} (1/10 scale) of the flow, within the same gate as Table 6's.

Test	What it shows	Tolerance	Measured	Commit
`Dyn4PH.TheStructureIsPortHamiltonianAtEverySlip`	J skew, R symmetric positive semi-definite with secants $S_i \geq 0$, ∇H against H's central differences, no supply without wheel forces (Sec. 6); 500 states each on flat ground at $\mu = 0.9$ and 0.6 and on a friction split with a bump, sprung and with rigid corners	exact for skew, symmetry and zero supply; least eigenvalue of R at least $-10^{-9}\|\mathsf{R}\|$; ∇H to 10^{-6} of its scale; at least 100 states past a peak	holds at all 2000 states; 267 of 500 past a peak on each ground; ∇H within 6.0×10^{-8}	`45dab152` (#850)
`Dyn4PH.TheVectorFieldIsThePlantsAtEverySlip`	(22) is the plant's vector field: a 10^{-5} s `dyn4_plant` step against `dyn4_ph_rhs`, 400 states per ground, forward and reversing, flat and on the split with a bump, with and without the grade force	10^{-3} of each row's scale (mg, mgL on yaw)	worst 1.5×10^{-4} (flat), 2.0×10^{-4} (split and bump)	`45dab152` (#850)
`Dyn4PH.StepBooksCloseToSecondOrder`	the books (24) close to second order per step on an 8 s weave past the peak, flat and over hills; the limiters are charged when braking to a stop in a turn; no lever with the centre of gravity on the ground	per-halving ratio in $(3, 5)$; summed residual below 5×10^{-3} of the flow for $h \leq 0.01$; lever exactly 0	ratios 3.92, 3.95, 3.98 (flat), 3.84, 3.91, 3.95 (hills); 5.0×10^{-4} and 1.0×10^{-3} at $h = 0.01$; braking: limiter -340 J, residual 7.3×10^{-5}	`45dab152` (#850)
`Dyn4PH.TheGradeForceIsMinusTheSpringsGradientInThePose`	with `grade_force`, f_{grade} is $-\partial H/\partial$(pose) (central differences) on the bump and on hills; zero without it	10^{-5} of mg (mgL on yaw)	within $3.6 \times 10^{-7} mg$ and $2.1 \times 10^{-8} mg$	`45dab152` (#850)
`Dyn4PH.TheGradeForcePutsTheTerrainInH`	with it, the grade line cancels the ground line to second order; up a 5% ramp the coast pays the climb from its speed, and without it the speed holds	per-halving ratio in $(3, 5)$; $\|\text{ground} + \text{grade}\|$ below 0.05 of the ground line; ramp within 1%, halving with h	ratios 4.00, 4.00, 4.00; ground $+3373$ J, grade -3372 J at $h = 0.01$; ramp: 0.986 m/s lost against 0.981	`45dab152` (#850)
`Dyn4.Golden`, `Dyn4ExtRollout.Golden`	`dyn4_plant`'s outputs and its rollout with an external force, unchanged by the form and the books	golden values	hold	`2e02c3eb` (#731), `c48d5a11` (#798)

Table 8: The four-wheel vehicle's pH form and books (libcvc's `nav_dyn_4w_test`), measured on `master` at `6cc8cfdc` (Table 17). The flow is the sum of the magnitudes of every line of the books.

Test	What it shows	Tolerance	Measured	Commit
`DynSpinPH.TheStructure IsPortHamiltonianAt EverySlip`, `Dyn4SpinPH.*`	J skew, R symmetric with longitudinal secants $K_w \geq 0$ and $w^\top \mathsf{R} w \geq 0$, $y^\top u$ the supplied less the braked, braked ≥ 0, no supply without torque (Sec. 7.1); 500 states at each of $\mu = 0.9, 0.3$ (four wheels: 2000 wheels)	exact for skew, symmetry and zero supply; $w^\top \mathsf{R} w \geq -10^{-12}\|\mathsf{R}\|$; at least 200 states past a peak (800 wheels)	holds; 420 of 500 states past the longitudinal peak at each μ; 1193 of 2000 wheels	`4b640693` (#982)
`DynSpinPH.TheVector FieldIsThePlantsAtEvery Slip`, `Dyn4SpinPH.*`	(26) is the plant's vector field: a 10^{-7} s plant step against the form	10^{-3} of mg (mgr_w on the wheels; four wheels, of each row's scale)	worst 2.5×10^{-4} ($\mu = 0.9$), 4.4×10^{-5} (0.3); four wheels 5.6×10^{-4}, 9.6×10^{-5}	`4b640693` (#982)
`DynSpinPH.TheForcesAre TheFieldLessThe Structure`	$\mathsf{J}\nabla H + Q$ is the field, and $\nabla H^\top Q$ the balance's rates: the forces a discrete-gradient step takes	10^{-12}	2.1×10^{-15} of mg; 1.3×10^{-14}	`4b640693` (#982)
`DynSpinPH.StepBooks CloseToSecondOrder`, `Dyn4SpinPH.*`	the books close to second order on a swerve past the lateral peak at $\mu = 0.6$, and on a launch at full drive on $\mu = 0.3$ that spins the wheels, then locks them; h from 0.01 to 0.00125 s	per-halving ratio in $(3, 5.5)$, or $(3, 10)$ on the launch; summed residual below 5×10^{-3} of the flow; peak slip ratio over 0.5	swerve 3.92, 3.96, 3.98; launch 5.52, 7.53, 6.64; 1.3×10^{-3} and 4.9×10^{-4} at $h = 0.01$; peak slip ratio 8.7. Four wheels: 3.91, 3.96, 3.98; 6.39, 7.37, 5.94; 1.5×10^{-3}, 5.2×10^{-4}; 9.0	`4b640693` (#982)
`DynSpin.SpinsOnLowGrip LocksUnderHardBraking RollsLikeDynPlant`	spins on low grip, locks braking past the peak, rolls like `dyn_plant` with light wheels, and accelerates as its effective mass predicts	slip ratios over 0.2; lock below -0.8; locked stop at least 2% longer; within 0.03 m; within 1% of the speed gained	3.57 front, 1.76 rear; -0.97; 34.8 m against 33.9 m; 0.002 m; 11.5369 against 11.5376 m/s	`4b640693` (#982)
`DynSpin.PlantGradient MatchesFiniteDifference`, `Dyn4Spin.*`	the plants' dual-number VJPs against central differences	10^{-5} relative	1.8×10^{-8}; 1.3×10^{-8}	`4b640693` (#982)
`DynSpinPH.Golden`, `Dyn4SpinPH.Golden`	the goldens of GRL-SNAM's twins (9 and 7 cases); the books' step is the plant's	golden values; bit for bit	hold	`4b640693` (#982)
`DynBikePlant.WheelSpin SpinsLocksAndTheBooks Close`	the live plant with wheel spin, on grip 0.2: it spins and locks, and the books (21) close with the slip line; off, the wheels roll at v_x/r_w and the slip line is zero	spin over 0.2, lock below -0.8; identity 10^{-9} of the flow; residual 5×10^{-3}	rear slip ratio up to 7.00, front down to -0.99; residual 6.8×10^{-5} of the flow	`4b640693` (#982)

Table 9: Wheel spin (Section 7.1; libcvc's `nav_dyn_spin_test` and `physics_dyn_bike_plant_test`), measured on `master` at `3d5c15c6`. A starred name is the single track's test's four-wheel twin. The slip ratio is σ_w/u_w; the flow is the sum of the magnitudes of every line of the books.

Test	What it shows	Tolerance	Measured	Commit
`DynPHFull.TheStructure IsSkewAndPoisson`	the structure of (12) is skew and satisfies the Jacobi identity (central differences); $\nabla_q V$ is V's gradient, and $-T^\top \nabla_q V$ is $-mg(\theta_\parallel, \theta_\perp, 0)$; 300 states on a hillside	exact skew; $e^\top \mathrm{J} e$ to 10^{-14} of its terms; Jacobi 10^{-8}; gradient $10^{-6}mg$; force 10^{-14}	1.1×10^{-16}; 3.1×10^{-9}; $1.6 \times 10^{-9}mg$; 3.5×10^{-17}	`b89992ef` (#987)
`DynPHFull.ThePoseUpdate IsTv`	the step's pose update is $q_1 = q_0 + h\,T(q_1)v_1$; against $T(q_0)v_0$ it is first order	10^{-14}; ratio per halving in $(1.7, 2.3)$	1.8×10^{-16} at 400 states; 0.176, 0.087, 0.043, 0.021 for $h = 0.04$ to 0.005 s	`b89992ef` (#987)
`DynPHFull.TheVector FieldIsTheFullFormsAt EverySlip`	(28) is the plant's vector field: a 10^{-5} s step at $\mu = 0.9$ and 0.6	10^{-3}	$\dot{q}$ within 6.1×10^{-5} of the speed, $\dot{p}$ within $1.65 \times 10^{-4}mg$	`b89992ef` (#987)
`DynPHFull.SideSlope BooksCloseWithGravityIn H`	closed loop across a 15% hillside for 6 s, the body slipping: with V in H the books close to second order; the momentum-only books, the plant without the side slope and the sines leave gaps that do not shrink with h	ratio in $(3, 5.5)$; summed residual below 2×10^{-3} of the flow; the gaps at least 30, 5 and 3 times it	ratios 4.04, 4.02, 4.01; 2.8×10^{-4} ($h = 0.02$) to 2.8×10^{-5} (0.0025); gaps 1.5–1.6×10^{-2}, 1.2–1.5×10^{-3}, 1.8–2.1×10^{-3}	`b89992ef` (#987)
`DynPHFull.TheWheelSpin PlantTakesTheSlope AcrossItsHeading`	the wheel-spin plant on the hillside: its books close	summed residual below 2×10^{-3} of the flow; ratio over 2.5	0.790, 0.198, 0.050 J per step; at most 8.7×10^{-5}	`b89992ef` (#987)
`Dyn4PHFull.TheStructure IsSkewAndPoissonAndThe GradeForceIsIts`	the four-wheel full structure is skew and satisfies the Jacobi identity; $\partial H/\partial g$ against H's differences; $-T^\top \partial H/\partial g$ is the grade force; 200 states	energy 10^{-12}; Jacobi 10^{-8}; gradient 10^{-6}; grade force 10^{-12}	1.7×10^{-16}; 9.1×10^{-11}; $1.6 \times 10^{-8}mgL$; 6.9×10^{-17}	`b89992ef` (#987)
`Dyn4PHFull.TheVector FieldIsTheFullFormsWith TheGradeForce`	with the grade force, the plant's field is the full form's over 12 coordinates; without it the momenta miss the slope's force; the pose step	10^{-3} of each row's scale; a gap over 10^{-2}; 10^{-14}	2.2×10^{-4}; 0.20; 1.6×10^{-16}	`b89992ef` (#987)
`DynBikePlant.SideSlope PullsDownhillAndBooks ThePotential`	the live plant with the side slope, driven straight across a 10% side slope for 6 s: pulled downhill, grade line $-\Delta V$; off, it runs straight with no grade	over 0.1 m; 10^{-9} of the flow; residual 5×10^{-3}; exact	0.196 m (0.187 with wheel spin); 515.187 J against $-mg\Delta z =$ 515.187 J; residual 8.0×10^{-5} (8.7×10^{-5}); off: 0	`b89992ef` (#987)
`DynPHFull.Golden`	the golden of GRL-SNAM's twin: 6 states on a hillside; the books' step is the plant's	golden values; bit for bit	holds	`b89992ef` (#987)

Table 10: The full (q, p) form and the side slope (Section 7.2; libcvc's `nav_dyn_ph_full_test` and `physics_dyn_bike_plant_test`), measured on `master` at `3d5c15c6`.

Test	What it shows	Tolerance	Measured	Commit
`DynContact.TheBarrierIs ItsEnergysGradientAnd TheDamperDissipates`	the barrier's wrench is $-\partial\Phi/\partial$(pose) (central differences) against a wall and a pillar; the damper's power is minus its dissipation	10^{-5} of the scale; 10^{-9}	within 5.2×10^{-10} over 165 poses in the skin	`2728bca8` (#983)
`DynContact.APairKeeps ItsMomentum`	between two vehicles, the forces and moments balance, and the barrier is $-\partial\Phi/\partial$(pose)	10^{-9}; 10^{-5}	balance over 168 touching pairs; barrier within 3.3×10^{-10}	`2728bca8` (#983)
`DynContact.TheImplicit DamperOnlyDissipates`	the implicit damper and friction: the rows' $\mathsf{R}_c v$ is $-W_d$, the solve holds, $W_d^\top v_d \le 0$, a pair balances, and without the barrier the step never raises the kinetic energy; 846 wall and 638 pair poses at $h = 0.01$, 0.1 and 1 s	10^{-12}; 10^{-9}; power $\le 10^{-9}$ of the scale; 10^{-9}; 10^{-12} of the kinetic energy	7.9×10^{-16}; 3.4×10^{-16}; holds; 1.8×10^{-15}; holds	`ce9aebaf` (#1004)
`DynContact.Golden`	the golden of GRL-SNAM's twin: the footprint, field and pair cases, and a plant step	golden values	holds	`2728bca8` (#983)

Table 11: Compliant contact's model (Section 7.3; libcvc's `physics_dyn_contact_test`), measured with libcvc #1004's implicit damper (Table 17). The gates and the golden are #983's; #1004 adds the implicit damper's.

Test	What it shows	Tolerance	Measured	Commit
`DynContactPlant.The BooksCloseToSecondOrder`	the live plant's books close to second order through 5 m/s impacts: a wall head on and at 30°, and a parked vehicle; substep 0.01 to 0.0025 s	identity 10^{-8} of the flow; ratio in $(3, 5.5)$; residual below 5×10^{-3} of the flow; exchange below 2×10^{-2} of it, falling at least 2.5 times per halving	head on 4.74, 4.20, 4.0×10^{-3} at 0.01; 30°: 4.10, 4.32, 2.8×10^{-3}; parked: 4.35, 4.19, 1.7×10^{-5}, exchange −242, −84.4, −20.3 J	`2728bca8` (#983)
`DynContactPlant.AFast ImpactAtTheDefault Substep`	at the default 0.01 s substep, each against the same run at 0.00125 s: the navigator driving into the navigation world's wall from 20 and 30 m, raw drives into it at 8.5, 10.4 and 12.1 m/s, an exact wall at 30° and a parked vehicle at 10.4 m/s	identity 10^{-8} of the flow; contact dissipation > 0; limiter no more than at 0.00125 s; residual below 5×10^{-3} of the flow; rebound within 0.02	navigator: 8.16×10^4 J (8.19×10^4), slowest −0.47 m/s at both, 1.3×10^{-3} and 3.9×10^{-3}; raw: rebound 0.050, 0.040, −0.018, as at 0.00125 s, residual 4.1×10^{-3}, 7.9×10^{-4}, 1.7×10^{-3}	`ce9aebaf` (#1004)
`DynContactPlant. ComparesWithJoltOn Impacts`	the same impacts in the rigid-body world, coasting: both stop at the wall, the parked vehicle moves, and the heading turns little	normal speed below half the approach's; over 0.5 m/s; under 0.2 rad	rebound 0.19 (compliant), 0.00 (rigid); at 30°, +0.3° and −2.8°; parked: 1.90 and 2.88 m/s against 2.27 and 2.28	`2728bca8` (#983)
`DynContactPlant.Reads NavsSignedDistanceGrid`	the field from the navigation world's grid is the wall's	10^{-5}	9.5×10^{-7}	`2728bca8` (#983)
`DynContactPlant.StopsAt AWallInSimWorldsField`	in the navigation world's own field, a vehicle at 5 m/s stops at a wall 25 m on	ϕ within 1 m; front between 24 and 25.4 m	$\phi = 5.000$ m 5 m out, gradient $(-1, 0)$; front at 24.53 m	`2728bca8` (#983)

Table 12: Compliant contact in the live plant (Section 7.3; libcvc's `physics_dyn_contact_test`), measured with libcvc #1004's implicit damper on the full-size preset (Table 17). The ratios are the mean per-step residual's fall per halving of the substep. Before #1004, on `3d5c15c6`, the head-on residuals per step were 3.23, 0.70 and 0.16 J, and at 30° the compliant plant turned +0.4° (−0.1° at friction 0.6); the gates are unchanged.

Test	What it shows	Tolerance	Measured	Commit
`PhSe2.MatchesJaxIn Double`, `PhSe2.MatchesJax InFloat`	the C++ model is the JAX reference's: every piece and one step at 16 states (a 64-feature, 1,691-parameter fit)	10^{-13} relative (double); 2×10^{-6} (float)	1.9×10^{-16}; 1.1×10^{-7}	#758
`PhSe2.RolloutMatchesJax`	a 25-step rollout matches	10^{-12}	3.1×10^{-15}	#758
`PhSe2.EnergyBalanceIs Exact`	(32) at every golden state; $y^\top u$ from `output()` is the supply	10^{-11}; 10^{-12}	holds	#758, `950154b` (#831)
`PhSe2.NoisePowerIsThe EnergyTheDiffusionAdds`	(33): antithetic Monte Carlo of H's mean gain over one 10^{-4} s step	4 standard errors +2%	3.585×10^{-6} J against 3.564×10^{-6} J predicted	`950154b` (#831)
`PhSe2.StepIsSecondOrder`	the Lie-IMEX step (34) is second order	ratio in $(3.5, 4.5)$ per halving	3.99, 3.99, 4.00	#758
`PhSe2.ReducesToThe LinearTireBicycle`	with the GPs off, the drift is the linear-tire bicycle with steering as an input	10^{-9}	holds	#758
`PhRff.FeaturesApproach TheMaternKernel`	(31) approximates the Matérn kernel	0.03	holds	#758
`PhSe2Train.Gradient MatchesJax`	the hand-written reverse pass is JAX's autodiff of (35)	10^{-10} of each leaf's largest entry	8.1×10^{-15} over 1,691 entries	#758
`PhSe2Train.Gradient MatchesFinite Differences`	the same against central differences on a small model	2×10^{-5}	5.6×10^{-6}	#758
`PhSe2Train.Optimizer MatchesOptax`	clip, Adam and Prodigy match the reference optimizer over 300 steps	10^{-12}	1.6×10^{-16}	#758
`PhSe2Train.ShortFit LowersTheLoss`	a 1,200-step fit to trajectories simulated from a second random model generalizes to held-out ones	held-out loss below 0.1×	$1.732 \to 0.077$ (22×)	#758

Table 13: The learned plant (libcvc `ph_se2_test`, `ph_se2_train_test`). libcvc #758 merged as `17adc93`, with the model in `0fa251b` and the trainer in `63651e8`.

Test	What it shows	Tolerance	Measured	Commit
`PhSe2.TheReferenceForm BooksSteeringAsPower`	in the reference form, steering alone supplies power (Sec. 8.6)	nonzero	share 0.134; supplies energy at 10 of 16 states	`b8ac9975` (#851)
`PhSe2.ModulatedSteering SuppliesNoPower`	(36): the balance holds; no supply at zero drive, whatever δ; steering still changes the motion	balance 10^{-11}; supply exactly 0	holds at 200 states; a 0.5 steer step changes the twist's rate by at least 0.18	`b8ac9975` (#851)
`PhSe2.ModulatedModel RoundTripsExactly`, `PhSe2. ModulatedStepIsSecond Order`	the `/2` file round-trips; models without modulations stay `/1`; the step stays second order	exact; ratio in $(3.5, 4.5)$	exact; 4.29, 4.08, 4.02	`b8ac9975` (#851)
`PhSe2Train.Modulated GradientMatchesFinite Differences`, `PhSe2Train. ModulatedShortFitLowers TheLoss`	the modulated reverse pass is the gradient; the modulated form trains	2×10^{-5}; held-out loss below $0.1\times$	3.9×10^{-7} over 68 entries; $1.417 \to 0.070$	`b8ac9975` (#851)
`PhSe2Train.BothFormsFit TheDynamicBicycle`	both forms fit the analytic plant's flights; only the reference form books steering power	held-out loss below $0.01\times$; share > 0 (reference), $= 0$ (modulated)	reference $8.52 \to 0.0006$, share 0.059, supplies energy at 33 of 250 steps; modulated $7.63 \to 0.0167$, share 0	`b8ac9975` (#851)
`PhSe2Train.Potential OptionsGradientsMatch FiniteDifferences`, `Ph Se2Train.Potential OptionsRoundTrip`	with no potential and with the terrain term (Sec. 8.7), the reverse pass is the gradient; the `/3` file round-trips	2×10^{-5} relative, or 10^{-9} absolute (central differences' rounding); exact	2.1×10^{-7} (none); the terrain term's worst relative 1.2×10^{-4}, within the absolute bound; exact	`4b640693` (#982)

Table 14: The learned plant's options (libcvc `ph_se2_test`, `ph_se2_train_test`). Steering (Section 8.6) is transfix/libcvc #851, measured on its branch, except `BothFormsFitTheDynamicBicycle`, measured on `master` at `6cc8cfdc` with #852's plant (Table 17). On the branch, before #852, that row measured share 0.056, 74 of 250 steps, and $7.63 \to 0.0092$. The potential's form is #982, measured on `master` at `3d5c15c6`.

Test	What it shows	Tolerance	Measured	Commit
`NavPotential.Surrogate EnergyIdentityPerStep`	(38): r_E is second order and $r_E - \hat{r}_E$ third, as h halves $0.02 \to 0.0025$, with damping and a port force	orders 2 ± 0.2 and 3 ± 0.3; $\lvert r_E \rvert \le$ bound at every step	1.99, 2.00, 2.00; 2.99, 2.94, 2.93	`09fa876` (#832)
`NavPotential.ClampWork ClosesTheBooks`	with a hard speed clamp, the books close only with $W_\times$ (and W_{clamp})	every clamped step within the bound; 90% fail without $W_\times$	213 of 213 close; all 213 fail without it	`09fa876` (#832)
`NavPotential.Unlogged ForceRaisesTheAlarm`	an unbooked port force fails the cheap bound	at least 95% of steps	229 of 240	`09fa876` (#832)
`NavPotential.Explicit EulerRaisesTheAlarm`	explicit Euler gains energy every step and fails the sharp check; semi-implicit meets it	sharp check above 0.3 of the bound (explicit), below 10^{-3} (semi-implicit)	0.50; 3.5×10^{-5}	`09fa876` (#832)
`NavPotential.SimWorld BooksTheContextWorkOf AGoalChange`	a goal change between ticks is context work; the books close across the tick; capturing changes no output	exact equality; byte-identical positions	holds (retargeted agents: -1.63, -3.27)	`09fa876` (#832)
`NavPotential.SimWorld BooksTheMinGapPushAs ProjectionWork`	`min_gap`'s push is projection work, zero for agents it does not move	nonzero for the pair, exactly zero otherwise	$+0.18$, $+0.59$; others 0	`09fa876` (#832)
`NavPotential.*ForcesAre MinusGradU*`, `Nav Potential.DriveForce*`	each call site's conservative force is $-\nabla U_{\text{nav}}$ (central differences)	site-specific	worst 5.7×10^{-4} relative on exact planes	`7d47b98` (#747)
`NavGeomRolloutGrad.Semi ImplicitIsTheDefault`	the geometry rollout and the multi-start loss default to semi-implicit Euler: the default loss is the semi-implicit one bit for bit, not the explicit one	exact	holds; the suite's gradchecks give penalties 2.050 (semi-implicit) and 2.230 (explicit) on the legacy start, 2.453 and 2.898 on sampled starts	`45cb239a` (#981)
`NavCoefTrain.Explicit EulerGradcheckMatches FiniteDifference`, `Nav CoefTrain.PointMass IntegratorsDiffer`	the coefficient trainer's explicit-Euler arm (the A/B's) has its own backward, held to central differences; the two arms are two rollouts, and the default is the semi-implicit one bit for bit	directional error 2×10^{-2}; exact	6.5×10^{-4} over 419 parameters; window loss 1.588633 against 1.591275	`45cb239a` (#981)
`NavMaterialTrain.Full ChainGradcheckExplicit MultiStart`	the material trainer's whole loss, with explicit Euler in its multi-start term: the gradient to the weights against finite differences	directional error 2×10^{-2}	2.2×10^{-4}	`45cb239a` (#981)

Table 15: The navigator's books (libcvc `nav_potential_test`), and its integrator's default (`nav_geom_rollout_grad_test`, `nav_coef_train_test`, `nav_material_train_test`; the #981 rows, measured on `master` at `352f1933`).

Test	What it shows	Tolerance	Measured	Commit
`test_dyn_bike_parity.py`	the torch plant and its autograd gradient are libcvc's (7 cases, a batch, braking distance)	10^{-9} relative	9 passed (also on #138's wheel-frame slip)	GRL-SNAM `d75eecc` (#121)
`test_dyn_bike_ph_parity.py`	the torch pH parts, rates and step books, and the books' autograd gradient, are libcvc's (8 cases, a batch, and second-order books on a swerve)	10^{-9} relative	18 passed; worst 7.1×10^{-14}	`2f76ab0` (#138)
`test_nav_potential_parity.py`	the torch U_{nav} is libcvc's; its forces are $-\nabla U$ by autograd	2×10^{-5}; 5×10^{-5} along the rollout	8 passed	`7e4195d` (#133)
`test_dyn4_parity.py`	the four-wheel torch plant is libcvc's	10^{-9}	14 passed (11 cases)	`5b8e21f` (#123)
`test_dyn4_ext_rollout_parity.py`, `test_material_ext_parity.py`	the dynamic rollouts with an external force, forward and reverse	10^{-4} forward, 10^{-3} gradients	22 passed, 2 skipped	`0e21085` (#136)
`test_dyn_spin_ph_parity.py`, `test_dyn4_spin_ph_parity.py`	the torch wheel-spin plants, their pH parts, rates and books, and the books' autograd gradient, are libcvc's (9 and 7 cases)	10^{-9} relative	20 and 17 passed; worst 4.5×10^{-15} and 7.3×10^{-16}, gradients 4.4×10^{-14} and 7.2×10^{-13}	`b6ddb801` (#146)
`test_dyn_contact_parity.py`	the torch contact (footprint, field, pair, a plant step with its port) is libcvc's; the barrier is minus its energy's autograd gradient; a pair balances	10^{-9} relative	12 passed; worst difference 0; 2.8×10^{-16} over 157 poses, 2.0×10^{-15} over 254 pairs	`b6ddb801` (#146)
`test_dyn_ph_full_parity.py`	the torch full form, ground overloads and side-slope books are libcvc's, with the books' gradient through the ground; skew and Jacobi by autograd; the pose update; the side-slope books close	10^{-9} relative	16 passed	`b6ddb801` (#146)

Table 16: GRL-SNAM's twins (CVC-Lab/GRL-SNAM `tests/`), against libcvc's goldens. GRL-SNAM numbers are its pull requests.

12 Limitations and future work

The four-wheel vehicle's form. Three choices in Section 6 are left as the plant makes them, since changing any of them changes the plant, its goldens and GRL-SNAM's `dyn4.py` twin:

- The lever is one-way. Making it reciprocal means adding the roll and pitch rates to each contact patch's lateral velocity.
- The grade force is off by default, so a vehicle climbs a hill for free unless its host turns it on.
- The dampers act on the body's own vertical rates, not on its rates relative to the ground, so a steady climb pays them too. The books show that as dissipation.

Without wheel spin, the four-wheel form and its books have no torch twin yet, and the twin does not have the grade force. The four-wheel plant has no compliant contact.

Structure-preserving integrators. Neither plant's default integrator keeps a discrete energy identity. Both are first order (analytic) or second order (learned) with a booked residual. The single track's opt-in discrete-gradient steps (Section 5.3) close their books exactly. Without wheel spin the step costs about ten times the default's and is not L-stable; with it, about twenty times, and its wheels' rows at the step's end cost their speeds an order of accuracy and book the scheme's own damping. `pycvc` binds neither, and GRL-SNAM has no twin of either. The four-wheel and learned plants have none.

The learned plant's evidence bound. The rollout objective does not fit the diffusion or an observation model. LieSPHGP's EKF-based ELBO is the next stage for `cvc::ph`. Beyond the synthetic comparison of Section 8.6, the fit's accuracy against the analytic plant is not reported here.

The learned plant's steering, upstream. The modulated form of Section 8.6 exists only in libcvc. It has no JAX golden, and whether LieSPHGP or the $SE(2)$ package adopts it is for their authors to decide.

The closed loop. The navigator commands the vehicle through a path-following controller, not an energy-shaping one, so the closed loop is not port-Hamiltonian, and the two sets of books of Section 9.4 stay separate. Interconnection and damping assignment for a car is a research problem: with two inputs and three velocity degrees of freedom, its matching condition fails in general [9].

Python access. The plant step's books and the navigator's point-mass books have torch twins (GRL-SNAM #138 and #139), and `pycvc` reads a running world's tick telemetry (libcvc #857). `pycvc.dyn_bike_plant` (libcvc #979, `781b9a2f`, released in 3.7.9) puts a navigation world on the live plant, optionally in a rigid-body world the caller builds, and reads its books per tick and since attach. GRL-SNAM's `capture` shows them while it drives (GRL-SNAM #143). libcvc #1002 (`1ded7bed`, released in 3.7.9) binds the options of Section 7 and the slip and compliant-contact lines of the books, with a ground height from a `pycvc.height_field` and a contact field copied from a `sim_world`. libcvc #1038 (`2434c33a`, released in 3.7.12) binds `contact_step_depth` (#1004). Its test runs Section 7.3's wall impact at the default 0.01 s substep, where #1002's ran it at 0.005 s: the contact dissipates 8.16×10^4 J, the speed clamp takes nothing, and the books leave 1.3×10^{-3} of the flow. `pycvc` does not bind `dyn_4w_plant` or `plant_host`.

12.1 Gaps raised in review, and the work on them

Review of this paper raised three gaps between the models it describes and the motion they are meant to capture. libcvc has since closed all three: the first two with opt-in extensions (Section 7), and the third with a semi-implicit default for the navigator (Section 9) and an opt-in discrete-gradient step for the plant (Section 5.3).

1. *Actuation.* Neither analytic plant had wheel spin: the drive and brake forces were set within the tires' grip, with no wheel speed or longitudinal slip in the state. The learned plant's training rows had no wheel speeds either, and its potential $V(x, y)$ depends on world position, so a fit can tie energy to where the vehicle drove rather than to how. libcvc #982 adds wheel-speed states with a (wheel torque, wheel speed) port and the longitudinal slip's secants in R, in both analytic plants and the live plant (Section 7.1); the rigid-body simulator's wheel speeds in its logs; and the option to drop V or replace it by the logged ground slope (Section 8.7). The defaults are unchanged.

2. *Collisions.* Neither pH form had contact. The live plants collide in a rigid-body world (Section 5.4), but by handing the model the world's state, with the energy that changes booked as contact, outside the form, and `cut_flights` dropped every window with a contact, an impact or a wheel off the ground. libcvc #983 adds compliant contact to the single track's form, with a barrier energy in H and the contact's damping and friction in R, entering as an external port, against the scene and between vehicles (Section 7.3); and the logs keep the contact windows on request. libcvc #1004 makes the contact's damper and friction implicit, so fast impacts close their books at the default substep. The four-wheel plant has no compliant contact, a learned plant can fit a contact's mean response but not where it is, and inside a thick wall the navigation world's field is flat (Section 7.3).

3. *Integration.* libcvc's native multi-start rollout of the navigator defaulted to explicit Euler (`geom_rollout_params::semi_implicit` was false), which gains energy at every step on the orbits of Section 9. libcvc #981 makes semi-implicit Euler the default, with an A/B comparison (Section 9). libcvc #978 adds an opt-in discrete-gradient step for the single-track plant, whose books close to the solve's tolerance (Section 5.3), and libcvc #1045 extends it to the wheel-spin plant. The defaults' books still keep a residual, and neither discrete-gradient step has Python access.

Recordings. A reel of GRL-SNAM's navigator on vehicle plants, with three successes and five failures, is published as a GRL-SNAM pre-release: `https://github.com/CVC-Lab/GRL-SNAM/releases/tag/plant-reel-2026-11`. Each clip is one `grl-snam capture` run of GRL-SNAM's shipped coefficient policy (`coef_sdf.pt`) on a city scene whose buildings are OpenStreetMap footprints (© OpenStreetMap contributors, ODbL). Its read-out shows the sideslip, the share of the tire's peak grip in use, the lateral acceleration and the energy books. Each clip but the first has a JSON file with its measured outcome. The plants are the policy's point-mass surrogate, the navigator's kinematic bicycle, the live plant of Section 5.4 and the rigid-body simulator's vehicle, both of these last two with the full-size preset.

- *Successes.* The surrogate reaches four goals, retargeted live, each within its trained radius of 240 m. The kinematic bicycle drives 296 m up a street and arrives within 15 m in 5.6 s; having no tires, it runs at the trained metric's speeds, up to 115 m/s. The rigid-body vehicle, on grip 0.3, drives 234 m around a block's corner and arrives within 15 m.

- *Failures.* The kinematic bicycle stalls at a dead end 700 m short: the policy's wall-following escape needs the point mass's sideways step. The rigid-body vehicle, on grip 0.9, circles at 7 m/s, because the wall-following target sits inside its turning circle. On grip 0.3 the navigator asks the live plant to corner at 0.5 g when its tires hold 0.3 g; it slides past the tires' peak, to 67° of slip, and pins itself against a wall for 530 ticks. The rigid-body vehicle spins out on grip 0.3, up to 90° off its path. With the map unknown, on grip 0.2 at up to 25 m/s, it hits a wall it cannot stop for.

What fails is the policy and the navigator's plan, not the plants. The policy was trained on the surrogate, where arrival is within 240 m, and does not transfer to a vehicle that must arrive within 15 m; the navigator caps cornering at 0.5 g whatever the grip. Over 24 random routes of 120–400 m in the same scene, it arrives on 3 with the surrogate, 3 with the kinematic bicycle, and none with the live plant or the rigid-body vehicle on grip 0.9 (GRL-SNAM's `docs/PLANTS.md`). The clips were rendered with GRL-SNAM #143 and libcvc #979.

A Reproducing the validation tables

A.1 What was run

Piece	Version
libcvc	`master` at `c1f104c0`; its `nav`, `ph` and `physics` sources and tests are identical to the v3.7.1 tag (`d8ca602b`). The #851 rows: its branch commit `6e7ab557`, cut from `master` at `cc8800a8`, whose `ph` sources and tests are those of its squash commit `b8ac9975`. The #852 rows (and every `nav_dyn_bike_test` and `physics_dyn_bike_plant_test` line below): its branch commit `1a0acc77`, on #848's head `d1a2a78e`, cut from `master` at `72e4c607`, whose `nav` and `physics` sources and tests are those of its squash commit `140bc2d0`. The #850, #869 and #873 rows, and `PhSe2Train.BothFormsFitTheDynamicBicycle`: `master` at `6cc8cfdc`, after v3.7.8. The #982, #983 and #987 rows: `master` at `3d5c15c6`, whose `nav`, `ph` and `physics` sources and tests are those of #987's squash commit `b89992ef`. The #981 and #1004 rows: `master` at `352f1933`, after v3.7.11, where every file the two PRs touched is as their squash commits `45cb239a` and `ce9aebaf` left it. The #978 rows: `master` at `f94a9b27`, where its files are as its squash commit `789d2f6e` left them. The #1045 rows: `master` at `fcfc4ee7`, #1045's squash commit. The #1038 numbers: its branch commit `a690b536`, on `master` at `f94a9b27`, whose `nav` and `physics` sources and tests, and `pycvc`'s physics binding and its test, are those of its squash commit `2434c33a`.
GRL-SNAM	`main` at `0e21085` (`3a38ae4` adds only this paper). The #138 rows: `main` at `2f76ab0`, #138's squash commit. The #146 rows: `main` at `41ddf84`, whose twins, tests and fixtures are those of #146's squash commit `b6ddb801`
Operating system	Windows 11 Pro 10.0.26100, x86-64
Compiler	MSVC 19.44 (Visual Studio 2022, toolset 14.44.35207), Release
Build tools	CMake 3.31.7, Ninja 1.12.1, cvcpkg 2.5.6
Dependencies	`cvcpkg install-deps` of libcvc's recipe: 38 packages for windows/x86_64/release/shared, among them Boost, googletest and HDF5
Python	3.11.13, torch 2.8.0+cpu, numpy 2.4.6, pytest 9.1.1

Table 17: The setup behind every measured value in Tables 3–16.

A.2 libcvc

From a libcvc checkout at v3.7.1 or later, in a Visual Studio 2022 x64 developer prompt (or after `vcvars64.bat`), the commands used were these, with the paths shortened:

```
cvcpkg install-deps cvcpkg\recipes\libcvc --prefix deps --config release
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=%CD%\deps ^
      -DCVC_BUILD_TESTS=ON -DCVC_BUILD_CVCGL=OFF -DCVC_ENABLE_CUDA=OFF ^
      -DCVC_BUILD_PYCVC=OFF -DCVC_BUILD_PYCVC_GL=OFF -DCVC_WITH_JOLT=ON
cmake --build build --target nav_dyn_bike_test nav_dyn_4w_test ^
      physics_dyn_bike_plant_test ph_se2_test ph_se2_train_test ^
      nav_potential_test nav_dyn_spin_test nav_dyn_ph_full_test ^
      physics_dyn_contact_test nav_geom_rollout_grad_test ^
      nav_coef_train_test nav_material_train_test nav_dyn_bike_dg_test
set PATH=%CD%\deps\bin;%CD%\build\bin;%PATH%
for %t in (nav_dyn_bike_test nav_dyn_4w_test physics_dyn_bike_plant_test
           ph_se2_test ph_se2_train_test nav_potential_test nav_dyn_spin_test
           nav_dyn_ph_full_test physics_dyn_contact_test nav_geom_rollout_grad_test
           nav_coef_train_test nav_material_train_test
           nav_dyn_bike_dg_test) do build\bin\%t.exe
```

The extra CMake flags only switch off what these tests do not need: the graphics library, CUDA and the Python bindings. `CVC_WITH_JOLT=ON` builds the rigid-body backend that `physics_dyn_bike_plant_test` links against, although the dynamic plant itself needs no physics world. If CMake finds no Python 3, add `-DPython3_EXECUTABLE=`*`path`*.

On Linux or macOS the same steps are

```
cvcpkg install-deps cvcpkg/recipes/libcvc --prefix deps --config release
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release -DCMAKE_PREFIX_PATH=$PWD/deps \
      -DCVC_BUILD_TESTS=ON -DCVC_BUILD_CVCGL=OFF -DCVC_ENABLE_CUDA=OFF \
      -DCVC_BUILD_PYCVC=OFF -DCVC_BUILD_PYCVC_GL=OFF -DCVC_WITH_JOLT=ON
cmake --build build --target nav_dyn_bike_test nav_dyn_4w_test \
      physics_dyn_bike_plant_test ph_se2_test ph_se2_train_test \
      nav_potential_test nav_dyn_spin_test nav_dyn_ph_full_test \
      physics_dyn_contact_test nav_geom_rollout_grad_test \
      nav_coef_train_test nav_material_train_test nav_dyn_bike_dg_test
for t in nav_dyn_bike_test nav_dyn_4w_test physics_dyn_bike_plant_test \
         ph_se2_test ph_se2_train_test nav_potential_test nav_dyn_spin_test \
         nav_dyn_ph_full_test physics_dyn_contact_test nav_geom_rollout_grad_test \
         nav_coef_train_test nav_material_train_test nav_dyn_bike_dg_test; do
  ./build/bin/$t; done
```

These were not run verbatim for this paper, since no Linux machine was available where it was written. libcvc's CI builds every test and runs it under CTest on Linux, macOS and Windows for each pull request. The tests are registered with CTest under their Google Test names, so `ctest -test-dir build -R 'Dyn|Ph|NavPotential|NavGeomRolloutGrad|NavCoefTrain|NavMaterialTrain'` runs these suites. That is a superset of the tables' rows.

A.3 What the libcvc tests print

Each test prints its measured values. These are the lines behind the tables, from the run of Table 17, with long lines cut at "...". Lines that name the vehicle preset are summarized rather than quoted.

Other compilers and platforms can differ in the last digits. The training runs (the held-out losses) can differ more, since a long optimization amplifies rounding. The tolerances in the tables are what the tests enforce.

```
nav_dyn_bike_test (23 tests, at #852's 1a0acc77):
  kinematic limit, 200 ticks (mean path 5.624), bike_step's commands through the plant:
    the kinematic bicycle's vehicle: within 0.01074 of its path (mean worst 0.00185)
  mu 0.9: 244 of 500 states have a tire past its peak
  mu 0.6: 244 of 500 states have a tire past its peak
  vector field vs dyn_plant at every slip, mu 0.9: worst |dp/dt - pH| = 1.62e-04 m g
  vector field vs dyn_plant at every slip, mu 0.6: worst |dp/dt - pH| = 1.06e-04 m g
  swerve past the peak, hdt 0.0200 s: ... mean |residual| per step 3.320e+00 J, |sum| / flow 2.24e-03
  swerve past the peak, hdt 0.0100 s: ... mean |residual| per step 8.664e-01 J, |sum| / flow 1.21e-03
  swerve past the peak, hdt 0.0050 s: ... mean |residual| per step 2.209e-01 J, |sum| / flow 6.25e-04
  swerve past the peak, hdt 0.0025 s: ... mean |residual| per step 5.593e-02 J, |sum| / flow 3.19e-04
  braking to a stop, hdt 0.0100 s: dH -1.084e+05 J, limiters -358.2 J, |sum residual| / flow 4.43e-06
  speed floor 5e-01 m/s: worst |floor power| / (dissipated + |supplied|) = 1.67e-16
  speed floor 5e-02 m/s: worst |floor power| / (dissipated + |supplied|) = 2.16e-16
  speed floor 5e-03 m/s: worst |floor power| / (dissipated + |supplied|) = 2.16e-16
  speed floor 1e-09 m/s: worst |floor power| / (dissipated + |supplied|) = 1.58e-16
  tangent vs central differences: worst 1.39e-07 of its scale
  (then the tangent's table by lateral g, at mu 0.9 and 0.6: 0% indefinite below 0.5 mu g)
  (on c1f104c0, before #852, 21 tests: the same lines, but braking -353.5 J and 7.23e-07,
   and speed floor 5.35e-04, 5.42e-06, 5.42e-08, 1.95e-16)
physics_dyn_bike_plant_test (3 tests, at 1a0acc77):
  RoboRacer, 30 s: drive 845.1 J, ..., floor 6.02e-16, ..., residual -1.49 (8.88e-04 of the flow)
  (and the full-size preset's line: floor -5.44e-15, residual 1.10e-04 of the flow; all agents
   reached in 11.5 s. On c1f104c0: RoboRacer 8.86e-04, full-size floor 1.14 J)
nav_dyn_4w_test (23 tests, on master at 6cc8cfdc; the Dyn4PH lines):
  flat, mu 0.9: 267 of 500 states have a tire past its peak; R's least eigenvalue on the twist
    4.68e-04 |R|; dH/dx within 6.0e-08 of H's differences
  (and 267 and 6.0e-08 at mu 0.6 and on the split and bump, sprung and with rigid corners)
  vector field vs dyn4_plant at every slip, flat, mu 0.6: worst |dx/dt - pH| = 1.53e-04 of its scale
  vector field vs dyn4_plant at every slip, split and bump: worst |dx/dt - pH| = 1.96e-04 of its scale
  (and 1.96e-04 on the split and bump with the grade force)
  flat, mu 0.6, swerving, hdt 0.0200 s: ... mean |residual| per step 2.331e+00 J, |sum| / flow 9.48e-04
  flat, mu 0.6, swerving, hdt 0.0100 s: dH -2.306e+05 J, supplied 4.566e+04, dissipated 2.771e+05,
    lever 1044, ground 0; mean |residual| per step 5.948e-01 J, |sum| / flow 5.01e-04; 214 of 800 ...
  flat, mu 0.6, swerving, hdt 0.0050 s: ... mean |residual| per step 1.504e-01 J, |sum| / flow 2.57e-04
  flat, mu 0.6, swerving, hdt 0.0025 s: ... mean |residual| per step 3.782e-02 J, |sum| / flow 1.30e-04
  hills, mu 0.6, swerving, hdt 0.0200 s: ... mean |residual| per step 5.124e+00 J, |sum| / flow 1.89e-03
  hills, mu 0.6, swerving, hdt 0.0100 s: ... ground 2868; mean |residual| per step 1.333e+00 J,
    |sum| / flow 1.03e-03
  hills, mu 0.6, swerving, hdt 0.0050 s: ... mean |residual| per step 3.409e-01 J, |sum| / flow 5.39e-04
  hills, mu 0.6, swerving, hdt 0.0025 s: ... mean |residual| per step 8.633e-02 J, |sum| / flow 2.76e-04
  braking to a stop, hdt 0.0100 s: ... limiters -339.8 J (|.| 709.1 J), |sum residual| / flow 7.27e-05
  centre of gravity on the ground, hdt 0.0050 s: lever 0 J, |sum residual| / flow 3.38e-04
  split and bump: the grade force is within 3.6e-07 m g of -dH/dpose (largest 0.024 m g)
  hills: the grade force is within 2.1e-08 m g of -dH/dpose (largest 0.018 m g)
  hills, swerving, hdt 0.0100 s: without the grade force ground 2868 J, grade 0; with it ground 3373 J,
    grade -3372, mean |ground + grade| per step 5.941e-01 J; ...
  (mean |ground + grade| per step 2.378e+00, 5.941e-01, 1.485e-01, 3.714e-02 J at hdt 0.02 to 0.0025)
  coasting up a 5% ramp for 2 s from 10 m/s, hdt 0.0025 s: without the grade force 10.0000 m/s,
    dH 2.625e+04 J (ground 2.636e+04, m g climb 2.624e+04); with it 9.0137 m/s (lost 0.9863,
    g s t 0.9807), dH -202.9 J (ground 2.499e+04, grade -2.499e+04)
```

```
ph_se2_test (14 tests; 18 with #851):
  one step, double vs JAX float64: worst |d| / (1 + |x|) 1.87e-16 over 16 states
  one step, float vs JAX float64: worst |d| / (1 + |x|) 1.08e-07
  25-step rollout, double vs JAX: worst 3.13e-15
  noise energy over 1e-04 s: 3.5851e-06 J measured (se 2.6e-08), noise_power h = 3.5643e-06 J
  substeps x2: error 2.570e-06 -> 6.442e-07 (x3.99)       (and x3.99, x4.00)
  [#851] steering's share of |y.u| at the golden states 0.134; it supplies energy at 10 of 16 ...
  [#851] modulated, substeps x2: error 3.028e-04 -> 7.055e-05 (x4.29)   (and x4.08, x4.02)
ph_se2_train_test (8 tests; 11 with #851):
  reference: 71 entries against central differences: worst relative 5.57e-06
  1,691 entries against JAX: worst |d| / max|leaf| 8.05e-15
  300 optimizer steps against optax: worst 1.58e-16 at 13 checkpoints
  held-out rollout loss 1.7320 -> 0.0773
  [#851] modulated: 68 entries against central differences: worst relative 3.87e-07
  [#851, on 6cc8cfdc] reference form on the dynamic bicycle: held-out rollout loss 8.5199 -> 0.0006;
         steering's share of |y.u| 0.059, supplying energy at 33 of 250 steps
  [#851, on 6cc8cfdc] modulated form on the dynamic bicycle: held-out rollout loss 7.6348 -> 0.0167; ...
  (on #851's branch, before #852's slip: share 0.056, 74 of 250 steps; modulated 7.6348 -> 0.0092)
nav_potential_test (16 tests):
  books, dt 0.0200 -> 0.0100: order of r_E 1.987, of r_E - r_hat_E 2.993
  books, dt 0.0100 -> 0.0050: order of r_E 1.997, of r_E - r_hat_E 2.943
  books, dt 0.0050 -> 0.0025: order of r_E 2.000, of r_E - r_hat_E 2.930
  clamp: 213 steps (76 in the barrier); |r_E|/bound worst 0.473; without W_cross 213 fail; ...
  unlogged port: 229/240 steps raise the alarm; smallest |r_E + W_port|/bound 0.44
  explicit Euler, 157 steps: ... (r_E - r_hat_E)/bound from 4.99e-01, |.| up to 4.99e-01; ...
  semi-implicit Euler, 157 steps: ... (r_E - r_hat_E)/bound from -3.54e-05, |.| up to 3.54e-05; ...
  agent 0 (retargeted): context work -1.6294 (without the goal change +0.0000), U 73.1821 -> 71.5528
  agent 2 (retargeted): context work -3.2729 (without the goal change +0.0871), U 74.8257 -> 71.5528
  min_gap push: projection work +0.1842, +0.5935
```

```
nav_dyn_spin_test (13 tests, on master at 3d5c15c6):
  (the full-size preset's wheels: r 0.47 m, J 12.0 / 12.0 kg m^2 an axle, B_x 46.2, C_x 1.202,
   peak at slip 0.080)
  full drive on mu 0.3 for 1 s from 5 m/s: 7.53 m/s, slip ratios 3.57 front, 1.76 rear (peak 0.080)
  braking at 8 m/s^2 from 20 m/s on mu 0.6: the front wheels' slip ratio reaches -0.97; stops in
    34.8 m (dyn_plant 33.9 m)
  a gentle weave for 5 s, the wheels' inertia at 1%: dyn_spin_plant within 0.002 m of dyn_plant, ...
  a 0.8 m/s^2 drive for 2 s from 10 m/s: 11.5369 m/s; m + sum J / r^2 = 2785 kg gives 11.5376, ...
  dyn_spin_plant_vjp vs central differences: worst 1.84e-08
  mu 0.9: 420 of 500 states have a wheel past its longitudinal peak; least w^T R w / |R| 0.00e+00
  (and the same at mu 0.3)
  vector field vs dyn_spin_plant at every slip, mu 0.9: worst 2.47e-04 of m g (m g r_w on the wheels)
  vector field vs dyn_spin_plant at every slip, mu 0.3: worst 4.38e-05 of m g (m g r_w on the wheels)
  J dH/dx + Q vs the field: worst 2.1e-15 of m g; dH/dx^T Q vs the rates: 1.3e-14 of its terms
  swerve past the peak, mu 0.6, hdt 0.0100 s: ... mean |residual| 9.359e-01 J, |sum| / flow 1.34e-03
  (mean |residual| 9.359e-01, 2.388e-01, 6.037e-02, 1.517e-02 J at hdt 0.01 to 0.00125)
  launch and lock, mu 0.3, hdt 0.0100 s: dH 489.4 J, drive 5.416e+05, brakes 1.1e+05,
    slip 4.143e+05, ...; peak |kappa| 8.67; mean |residual| 2.054e+00 J, |sum| / flow 4.87e-04
  (mean |residual| 2.054e+00, 3.719e-01, 4.941e-02, 7.441e-03 J at hdt 0.01 to 0.00125)
  (four wheels: 1193 of 2000 wheels past their longitudinal peak at mu 0.9 and 0.3; vector field
   worst 5.64e-04 and 9.59e-05 of its scale; swerve 9.701e-01, 2.478e-01, 6.261e-02, 1.574e-02 J,
   |sum| / flow 1.50e-03 at 0.01; launch and lock 2.000e+00, 3.131e-01, 4.249e-02, 7.152e-03 J,
   5.19e-04, peak |kappa| 8.95; dyn4_spin_plant_vjp vs central differences: worst 1.28e-08)
nav_dyn_ph_full_test (8 tests, on master at 3d5c15c6):
  300 states on the hillside: |e^T J e| / sum |terms| <= 1.1e-16; Jacobi defect <= 3.1e-09;
    dV/dq within 1.6e-09 m g of the exact gradient; -T^T dV/dq within 3.5e-17 of the slopes' force
  dyn_plant's pose step vs hdt T(q1) v1, 400 states: worst 1.8e-16
    hdt 0.040 s: |(q1 - q0) / hdt - T(q0) v0| = 1.759e-01   (then 8.696e-02, 4.316e-02, 2.149e-02)
  full vector field vs dyn_plant at every slip, mu 0.9: worst |dq/dt - T v| = 6.14e-05 of the
    speed, |dp/dt - pH| = 1.65e-04 m g   (mu 0.6: 4.06e-05, 1.10e-04)
  hdt 0.0200 s: gravity in H: dV 1.399e+05 J, body slip up to 0.051 rad (mean vy -0.263 m/s),
    mean |residual| per step 3.701e+00 J, |sum| / flow 2.77e-04; momentum-only books:
    |sum| / flow 1.53e-02; plant without the side slope: 1.52e-03; with the sines: 2.05e-03
  (hdt 0.01, 0.005, 0.0025 s: 9.166e-01, 2.278e-01, 5.676e-02 J; 1.24e-04, 5.83e-05, 2.83e-05;
   momentum-only 1.56e-02, 1.57e-02, 1.58e-02; without 1.31e-03, 1.21e-03, 1.17e-03;
   sines 1.90e-03, 1.84e-03, 1.81e-03)
  wheel spin on the hillside, hdt 0.0100 s: mean |residual| per step 7.902e-01 J,
    |sum| / flow 8.67e-05   (then 1.980e-01, 4.954e-02 J)
  dyn4, 200 states on the hillside: |e^T J e| / sum |terms| <= 1.7e-16; Jacobi defect <= 9.1e-11;
    dH/dg within 1.6e-08 m g L of H's differences; -T^T dH/dg within 6.9e-17 of dyn4_grade
  dyn4 full vector field vs dyn4_plant with grade_force, 300 states on the hillside: worst 2.24e-04
    of its scale; without grade_force the momenta miss up to 2.04e-01 of theirs; pose step vs
    hdt T(q1) v1 1.6e-16
```

```
physics_dyn_contact_test (9 tests, on master at 352f1933; the vehicle is the full-size preset):
  165 poses in the skin: the barrier's wrench within 5.22e-10 of -dPhi/dpose
  168 touching pairs: forces and moments balance; the barrier within 3.33e-10 of -dPhi/dpose
  846 wall and 638 pair poses: the rows' R_c v within 7.93e-16 of -Wd; the solve within 3.42e-16;
    the pair's forces and moments balance within 1.76e-15
  wall, head on, substep 0.0100 s: ... dissipated 2.882e+04 J, exchange 0 J; mean |residual| per
    step 3.446e+00 J, |sum| / flow 3.96e-03   (then 7.267e-01, 1.732e-01 J)
  wall, 30 degrees, substep 0.0100 s: ... dissipated 2.494e+04 J, exchange 0 J; mean |residual|
    per step 1.180e+00 J, |sum| / flow 2.80e-03   (then 2.875e-01, 6.655e-02 J)
  (into a parked vehicle of the same preset: exchange -242, -84.4, -20.3 J; mean |residual|
   3.775e-03, 8.677e-04, 2.072e-04 J; |sum| / flow 1.71e-05 at 0.01)
  the navigator from -20 m, at 7.77 m/s: slowest -0.47 m/s (-0.47 at 0.00125 s); contact_dissipated
    8.155e+04 J (8.191e+04), limiter 0 J, |residual| / flow 1.33e-03 (2.58e-04)
  the navigator from -30 m, at 7.99 m/s: slowest -0.47 m/s (-0.48 at 0.00125 s); contact_dissipated
    8.422e+04 J (8.497e+04), limiter 0 J, |residual| / flow 3.85e-03 (1.69e-04)
  sim_world's wall, head on at 8.45 m/s: rebound 0.050 (0.050 at 0.00125 s), dissipated 9.427e+04 J
    (9.51e+04), limiter 0 J (0); |residual| / flow 4.14e-03 (1.07e-04)
  (at 10.36 and 12.13 m/s: rebound 0.040 (0.039) and -0.018 (-0.018), dissipated 1.446e+05 and
   1.996e+05 J, |residual| / flow 7.94e-04 and 1.74e-03; a wall at 30 degrees at 10.40 m/s:
   rebound 0.088 (0.090), limiter -1.53e+04 J (-1.83e+04), 3.26e-03; into the parked vehicle at
   10.38 m/s: rebound -0.443 (-0.444), 1.07e-05)
  impact             plant   v before   v after   rebound    heading  other after
  wall, head on      dyn        4.80      0.89      0.19     -0.0 deg      0.00 m/s
  wall, head on      Jolt       4.56      0.01      0.00     -0.0 deg      0.00 m/s
  wall, 30 degrees   dyn        4.70      0.90      0.19      0.3 deg      0.00 m/s
  wall, 30 degrees   dyn .6     4.70      0.72      0.15     -0.5 deg      0.00 m/s
  wall, 30 degrees   Jolt       4.46      0.04      0.01     -2.8 deg      0.00 m/s
  (into the parked vehicle: dyn 5.06 -> 1.90, the other 2.88 m/s; Jolt 4.82 -> 2.27, the other 2.28)
  the grid's field against the wall's: worst 9.54e-07
  sim_world's field 5 m from the wall: phi 5.000 m, gradient (-1.000, 0.000)
  (and the vehicle's front reached x = 24.53 m, the wall at 25 m; slowest -0.50 m/s)
  (on master at 3d5c15c6, before #1004, 7 tests: head on 3.227e+00, 7.040e-01, 1.600e-01 J;
   30 degrees 1.099e+00, 2.651e-01, 6.413e-02 J; heading 0.4 and -0.1 deg; front at 24.50 m)
physics_dyn_bike_plant_test (10 tests, on master at 3d5c15c6; the #982 and #987 lines):
  (the full-size preset on mu 0.2 with wheel spin: rear slip ratio up to 7.00, front down to -0.99;
   slip 2.38e+06 J, residual 6.82e-05 of the flow)
  6 s across a 10% side slope: the centre of gravity 0.196 m downhill, heading -0.003 rad, body
    slip -0.039 m/s; grade 515.187 J against - m g dz = 515.187 J; residual 7.98e-05 of the flow;
    off: 0.00e+00 m
  (with the wheels spinning: 0.187 m, 490.241 J against 490.241 J, 8.72e-05)
  (the #869 and #873 lines as on 6cc8cfdc, and TheBooksClose's residuals 1.68e-04 and 6.45e-04)
ph_se2_train_test (13 tests, on master at 3d5c15c6; the #982 lines):
  no potential: 68 entries against central differences: worst relative 2.08e-07
  terrain potential: 68 entries against central differences: worst relative 1.16e-04

nav_geom_rollout_grad_test (16 tests, on master at 352f1933; the #981 lines):
  [geom-multistart legacy explicit] L=2.22984 |g|=261.4329 dir_rel=1.648e-04 checked=11 ...
  [geom-multistart legacy semi] L=2.04984 |g|=276.6617 dir_rel=1.385e-04 checked=11 ...
  [geom-multistart sampled explicit] L=2.89833 |g|=258.9400 dir_rel=1.323e-04 checked=12 ...
  [geom-multistart sampled semi] L=2.45327 |g|=280.5916 dir_rel=1.273e-04 checked=12 ...
nav_coef_train_test (30 tests, on master at 352f1933; the #981 lines):
  [gradcheck explicit] params=419 |g|=0.0118 dir_rel=6.473e-04 checked=15 worst_rel=1.612e-02
  window loss: semi-implicit 1.588633, explicit 1.591275
nav_material_train_test (18 tests, on master at 352f1933; the #981 line):
  [material-train-e2e legacy explicit] L=42.9632 |g|=241.928 dir_rel=2.170e-04 checked=232 ...
nav_dyn_bike_dg_test (7 tests, on master at f94a9b27):
  mu 0.9: worst |rate - dyn_ph_rhs| = 6.72e-16 of m g        (mu 0.6: 1.11e-15)
  swerve past the peak, hdt 0.0600 s: discrete gradient worst |residual| 1.08e-10 J (3.5e-14 of
    the step's flow), Newton 3.99 steps (max 4); dyn_plant mean |residual| 2.513e+01 J
  (hdt 0.02 to 0.0025 s: 1.34e-10, 1.54e-10, 1.32e-10, 1.69e-10 J (1.3e-12 of the flow);
   dyn_plant 3.320e+00, 8.664e-01, 2.209e-01, 5.593e-02 J)
  braking to a stop, hdt 0.0600 s: dH -1.084e+05 J, limiters -3146 J, worst |residual| 2.46e-11 J
    (and 4.84e-11, 5.91e-11 J at 0.01, 0.005 s)
  discrete gradient, hdt 0.040 s: error at 2 s 5.318e-03   (then 3.92x, 3.97x, 4.03x smaller)
  dyn_plant, hdt 0.040 s: error at 2 s 4.544e-01   (then 1.98x, 2.04x, 2.13x smaller)
  (coasting, 36 cases of tires, step and blend: worst step gain +0.0e+00 of H in every one)
  discrete gradient, 1 x 0.06 s: after 12 s r 0.30709 rad/s against the kinematic 0.30717 (-0.02%)
  hdt 0.06 s, per step: dyn_plant 300 ns, discrete gradient 3125 ns (10.4x)
  hdt 0.01 s, per step: dyn_plant 296 ns, discrete gradient 2823 ns (9.5x)
```

A.4 GRL-SNAM

From a GRL-SNAM checkout of `main`, with torch installed and no other setup (the parity tests need the copied goldens, not `pycvc`):

```
python -m pytest -v -p no:cacheprovider tests/test_dyn_bike_parity.py \
    tests/test_nav_potential_parity.py tests/test_dyn4_parity.py \
    tests/test_dyn4_ext_rollout_parity.py tests/test_material_ext_parity.py
```

On Windows, run it from the checkout's root with `PYTHONPATH` set to it (`set PYTHONPATH=.`). The expected summary is `53 passed, 2 skipped`. The two skips are `test_material_ext_parity.py`'s `pycvc` checks, which skip when the installed `pycvc` lacks the material external-force input. As GRL-SNAM's guide warns, a skipped test is not a verified one: read the `-v` listing. From `2f76ab0` (#138) on, add `tests/test_dyn_bike_ph_parity.py`, the pH twin's 18 tests, which all pass. From `b6ddb801` (#146) on, add `tests/test_dyn_spin_ph_parity.py`, `tests/test_dyn4_spin_ph_parity.py`, `tests/test_dyn_contact_parity.py` and `tests/test_dyn_ph_full_parity.py`, the twins of Section 7: 65 tests, which all pass. On `main` at `41ddf84` the ten files give `136 passed, 2 skipped`, the same two skips.

B Equation-to-code map

References

[1] A. S. Ellendula, Y. Wang, M. Nguyen, and C. Bajaj. GRL-SNAM: Geometric reinforcement learning with path differential Hamiltonians for simultaneous navigation and mapping in unknown environments. arXiv:2601.00116, December 2025.

[2] L. McLennan, T. Rapolu, and C. Bajaj. Stochastic port-Hamiltonian Gaussian processes for learning from noisy dynamics on Lie groups. Manuscript, CVC, The University of Texas at Austin, October 2026; code `CVC-Lab/LieSPHGP`.

[3] H. B. Pacejka. *Tire and Vehicle Dynamics*, 3rd edition. Butterworth-Heinemann, 2012.

[4] R. Rajamani. *Vehicle Dynamics and Control*, 2nd edition. Springer, 2012.

[5] A. Rahimi and B. Recht. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems 20*, 2007.

[6] J. E. Marsden and T. S. Ratiu. *Introduction to Mechanics and Symmetry*, 2nd edition. Springer, 1999.

[7] A. van der Schaft and D. Jeltsema. Port-Hamiltonian systems theory: An introductory overview. *Foundations and Trends in Systems and Control*, 1(2–3):173–378, 2014.

[8] A. J. van der Schaft and B. M. Maschke. On the Hamiltonian formulation of nonholonomic mechanical systems. *Reports on Mathematical Physics*, 34(2):225–233, 1994.

[9] R. Ortega, A. van der Schaft, B. Maschke, and G. Escobar. Interconnection and damping assignment passivity-based control of port-controlled Hamiltonian systems. *Automatica*, 38(4):585–596, 2002.

Concept	libcvc	GRL-SNAM
The plant's step, Sec. 3, 5	`diff::dyn_plant`, `dyn_plant_vjp` (`nav/detail/dyn_bike.h`)	`grl_snam.dyn_bike.dyn_plant`
Secant stiffness, Lemma 1	`dyn_secant_w`	`dyn_bike._secant_w`
H, p, J, (2)	`dyn_ph_hamiltonian`, `dyn_ph_momentum`, `dyn_ph_J`	`dyn_bike.dyn_ph_hamiltonian` (#138)
R, G, u, f_ε, (9)–(11)	`dyn_ph_R`, `dyn_ph_G`, `dyn_ph_input`, `dyn_ph_floor`; all parts in `dyn_ph_parts_at`	`dyn_bike.dyn_ph_parts_at` (#138)
The vector field (13); the balance (14)	`dyn_ph_rhs`; `dyn_ph_rates_at` (struct `dyn_ph_rates`)	the balance: `dyn_bike.dyn_ph_rates_at` (#138)
The tangent (15), (16)	`dyn_ph_tangent_D`	—
The step's books (18)	`dyn_plant_books` (struct `dyn_ph_books`); `dyn_step_free`	`dyn_bike.dyn_plant_books`; `dyn_plant(..., with_free=True)` (#138)
The discrete-gradient step (19), Sec. 5.3	`dyn_plant_dg`, `dyn_plant_dg_books` (`nav/detail/dyn_bike_dg.h`, #978)	—
With wheel spin (20), Sec. 5.3	`dyn_spin_plant_dg`, `dyn_spin_plant_dg_books`, `dyn_spin_dg_options` (`nav/detail/dyn_bike_dg.h`, #1045)	—
The live plant (21)	`cvc::physics::dyn_bike_plant`; `plant_host "dynamic"`; the options `wheel_spin`, `side_slope`, `contact`	`pycvc.dyn_bike_plant` (libcvc #979; the options #1002)
The four-wheel form (22)–(24), Sec. 6	`dyn4_ph_hamiltonian`, `dyn4_ph_J`, `dyn4_ph_R`, `dyn4_ph_G`, `dyn4_ph_lever`, `dyn4_ph_grade`, `dyn4_ph_rhs`, `dyn4_ph_rates_at`; `dyn4_plant_books` (`nav/detail/dyn_4w.h`, #850)	— (the plant: `grl_snam.dyn4`)
Wheel spin (25)–(27), Sec. 7.1	`dyn_spin_plant`, `dyn_spin_ph_J`, `dyn_spin_ph_R`, `dyn_spin_ph_G`, `dyn_spin_ph_rates_at`, `dyn_spin_plant_books`; `dyn4_spin_plant`, `dyn4_spin_plant_books` (#982)	`dyn_bike.dyn_spin_plant`, `dyn_spin_plant_books`; `dyn4.dyn4_spin_plant`, `dyn4_spin_plant_books` (#146)
The full form and the side slope (28), Sec. 7.2	`dyn_ph_T`, `dyn_ph_J_full`, `dyn_ph_rhs_full`, `dyn_plant_ground`, `dyn_plant_books_ground`; `dyn4_ph_J_full`, `dyn4_ph_rhs_full` (#987)	`dyn_bike.dyn_ph_rhs_full`, `dyn_plant_ground`, `dyn_plant_books_ground` (#146); four wheels: —
Compliant contact (29), Sec. 7.3	`nav/detail/dyn_contact.h`: `dyn_contact_field`, `dyn_contact_pair`; `dyn_bike_plant::sdf_field`; `cut_flights(..., keep_contact)` (#983); the implicit step `dyn_contact_implicit` and `dyn_bike_plant_options::contact_step_depth` (#1004)	`grl_snam.dyn_contact`; `dyn_plant(..., wrench=...)` (#146)
External force through the controller	`diff::ext_in`, `bike_control`; `nav/ext_rollout.h`	`grl_snam.ext_rollout`, `material_ext`

Table 18: Where each equation lives: the analytic plants. "—" marks C++-only pieces.

Concept	libcvc	GRL-SNAM
The learned plant (30)	`cvc::ph::se2_vehicle`, `se2_model`; its golden from the $SE(2)$ package built on `CVC-Lab/LieSPHGP`	—
Steering as a modulation (36)	`se2_setup::modulation_dim`, `cvc-ph-se2/2`, `cvc ph-train -steer-modulates` (#851)	—
The potential's form, Sec. 8.7	`se2_setup::potential`, `cvc-ph-se2/3`, `cvc ph-train -potential -inputs`, `cvc plant-log -wide` (#982)	—
Output, noise power, (32)–(33)	`se2_vehicle::output`, `mass`, `noise_power`	—
RFF features (31); SPD levels	`cvc::ph::rff`; `spd_from_levels` (`mat3.h`)	—
Lie-IMEX step (34)	`se2_vehicle::step`, `interval_step`, `rollout`	—
Rollout objective (35)	`se2_rollout_objective`, `se2_optimizer`; `cvc ph-train`	—
U_{nav}, (3)	`nav/detail/nav_potential.h`, `detail::material_energy`	`grl_snam.nav_potential`
Per-step identity (38); checks	`book_step`, `energy_residual_bound`; `rollout_energy`'s book fields	`nav_potential.book_step`, `energy_residual_bound`; `integrate_surrogate_material(energy=...)` (#139)
The geometry rollout's integrator, Sec. 9	`geom_rollout_params::semi_implicit`, `multi_start_params::semi_implicit` (true since #981); `train_config::point_mass`; `pycvc nav_material_trainer_create(..., ms_semi_implicit)`	`material_train_native` (libcvc's default); `material_train_torch` (explicit by default)
Context and projection work (39)	`drive_telemetry::potential_end`, `projection_work`, `context_work` (`sim_world`)	read via `pycvc nav_sim_world_drive_telemetry` (libcvc #857)

Table 19: Where each equation lives: the learned plant and the navigator.